PPS · Práctica 1: Git, GitHub y Docker

Sigue la demostración paso a paso. Después de cada operación, comprueba el resultado y anota qué cambió. No compartas contraseñas, tokens ni códigos de acceso; el repositorio debe permanecer privado.

1. Guardar versiones

1. Extrae el proyecto inicial y abre una terminal en la carpeta con index.html.
2. Comprueba `git --version`, `docker version` y `gh --version`. Si Docker no muestra cliente y servidor, avisa al profesor.
3. Inicializa Git en main y configura nombre/correo solo para este repositorio. Revisa el estado antes de añadir archivos.
4. Añade index.html, .gitignore, .dockerignore y LEEME.txt. Revisa `git diff --cached`, crea el primer commit y comprueba `git log --oneline`.
5. Cambia una línea de LEEME.txt; revisa `git diff`, añade solo ese archivo y crea un segundo commit.

2. Publicar y revisar

1. Crea un repositorio GitHub privado y vacío llamado pps-arranque; conecta tu copia y publica main.
2. Crea la rama mejora-portada, modifica el título de la página y publica la rama.
3. Abre una pull request hacia main. Revisa Files changed, incorpora una observación del profesor o compañero y fusiona la PR. Actualiza main local.

3. Construir y ejecutar

1. Escribe el Dockerfile siguiendo la demostración. Identifica qué aporta la imagen base, qué copia COPY y qué hace el comando de arranque.
2. Construye la imagen pps-web:v1. Añade el Dockerfile a Git, crea un commit y publícalo.
3. Ejecuta un contenedor con nombre y puerto libres. Abre la web en localhost; muestra `docker ps` y `docker logs`. Distingue el puerto del equipo del puerto del contenedor.
4. Cambia el texto visible a Version 2. Predice qué ocurrirá. Comprueba que editar el archivo y reiniciar no actualiza la imagen; reconstruye, recrea el contenedor y verifica Version 2.

4. Reproducir y entregar

1. Clona el repositorio en una carpeta nueva, construye desde esa copia y comprueba la versión.
2. Completa ficha-entrega.txt. Entrega URL privada del repo y de la PR, historial, captura del sitio y los logs, y tus respuestas.

Comprueba que puedes explicar

- Diferencia entre commit, push, pull request y merge.
- Diferencia entre imagen y contenedor; entre build, run y restart.
- Por qué editar un archivo no cambia el contenedor hasta reconstruir y crearlo de nuevo.
- Qué información aportan git status, git log, docker ps y docker logs.

Si algo falla, registra el mensaje, predice una causa y comprueba una cosa cada vez. No hagas público el repositorio ni elimines contenedores ajenos.

Índice

1. PPS · Práctica inicial — Git, GitHub y Docker
2. PPS · UT1 — Código, modelos de ejecución y pruebas de software
3. PPS · UT2 — Seguridad del lenguaje y del entorno de ejecución
4. PPS · UT3 — Nivel de verificación de seguridad: OWASP ASVS y superficie de riesgo
5. PPS · UT4 — Vulnerabilidades web I: entrada, inyección, sesión y control de acceso
6. PPS · UT5 — Vulnerabilidades web II: credenciales, bastionado del servidor y contramedidas
7. PPS · UT6 — Seguridad en aplicaciones para dispositivos móviles
8. PPS · UT7 — DevSecOps: control de versiones, construcción e integración continua
9. PPS · UT8 — Despliegue automatizado, resiliencia y recuperación ante desastres

PPS · UT1 — Código, modelos de ejecución y pruebas de software

9 horas · RA1 (a, b, c, d) · Servicio SOC: Prevención — calidad del software · Evidencias `PPS-E1`, `PPS-E2`

Qué vamos a conseguir

Leer código y demostrar con pruebas qué comportamiento garantiza.

Antes de empezar

Herramientas: Python, pytest y coverage; C y Java para lectura comparada.

Consulta la guía de instalación, la guía de herramientas y las precisiones del curso. Prepara una carpeta propia para resultados; conserva intactos los datos de partida.

Ruta de trabajo

1. Abre los tres validadores de laboratorio y marca entrada, transformación y operación sensible. Copia la línea exacta que fundamenta cada observación.
2. Predice el resultado con una entrada normal, una vacía y una fuera del rango. Distingue error de sintaxis, de ejecución y de lógica.
3. Trabaja en miniproyecto y ejecuta la suite inicial. Conserva la salida antes de modificar código.
4. Añade una aserción que compruebe el valor y otra que compruebe el rechazo de una entrada inválida. Una aserción siempre verdadera no verifica el programa.
5. Introduce un cambio controlado que reproduzca el fallo en una copia; comprueba que el test lo detecta, deshaz ese cambio y repite la suite.
6. Mide la cobertura y explica qué comportamiento importante sigue sin probarse. Entrega la tabla de lectura y las pruebas con su salida.

Cómo comprobar que está hecho

Suite inicial reproducible; test que falla con el defecto y pasa con la corrección; interpretación de cobertura.

> Que todos los tests pasen solo demuestra las propiedades que realmente comprueban. No equivale a ausencia de vulnerabilidades.

Desarrollo y fichas

- Explicación completa de la unidad.

Entrega y revisión

Los nombres y formatos están en el cuaderno de entregables. Antes de entregar, comprueba alcance, pasos reproducibles, resultados, interpretación y límites. Identifica tu trabajo por número de lista, módulo y evidencia.

Incluye una dificultad que hayas resuelto y explica por qué la solución funciona. Si utilizas una salida de ejemplo, declárala; no la presentes como ejecución propia.

PPS · UT1 — Código, modelos de ejecución y pruebas de software

9 horas · RA1 (a, b, c, d) · Servicio SOC: Prevención — calidad del software · Evidencias `PPS-E1`, `PPS-E2`

---

1. Objetivo de la unidad

Un analista de puesta en producción segura no escribe todo el software que revisa, pero tiene que **leerlo, probarlo y decidir si es apto para producción**. Esta unidad establece la base: cómo se ejecuta el código según el lenguaje, qué partes del fuente hay que saber leer y cómo se prueba de forma que un fallo no llegue al usuario. Todo lo que viene después —ASVS, corrección de vulnerabilidades, canalización— se apoya en saber distinguir un programa correcto de uno que solo *parece* correcto.

2. Contenido

2.1 Lenguajes: compilados, interpretados y de máquina virtual

Modelo	Ejemplos	Qué se despliega	Consecuencia de seguridad
Compilado a nativo	C, C++, Rust, Go	Binario máquina	Sin seguridad de memoria salvo Rust/Go; superficie en el propio binario (desbordamientos, cadena de formato)
Interpretado	Python, PHP, Ruby, JavaScript (Node)	Código fuente + intérprete	El fuente viaja al servidor: secretos y lógica quedan expuestos si el repositorio o la imagen se filtran
Máquina virtual	Java (JVM), C# (.NET)	Bytecode	Decompilable con facilidad (`jadx`, `dnSpy`); no confiar en la ofuscación como control

Regla operativa: el modelo de ejecución determina *dónde* está el riesgo. En un servicio Python, el `.py` está en el servidor; en un binario Go, el riesgo es de memoria y de dependencias enlazadas.

2.2 Elementos del código fuente que hay que saber leer

- **Puntos de entrada**: dónde entra dato del exterior (parámetros HTTP, ficheros, variables de entorno, argumentos). Es la superficie de ataque.
- **Puntos de salida (sinks)**: dónde ese dato acaba (consulta SQL, comando de shell, respuesta HTML, ruta de fichero). Es donde se materializa la vulnerabilidad.
- **Flujo entre ambos**: si un dato de entrada llega a un sink sin validación ni codificación, hay vulnerabilidad. Este es el modelo mental de todo el análisis estático.
- Dependencias, gestión de errores y gestión de secretos.

2.3 Pruebas de software

Tipo	Qué comprueba	Herramienta ejemplo
Unitaria	Una función en aislamiento	`pytest`, `JUnit`
Integración	Varias piezas juntas	`pytest`, `testcontainers`
Regresión	Que un fallo corregido no vuelve	La propia suite en CI
Cobertura	Qué porcentaje de líneas ejecuta la suite	`coverage.py`, `JaCoCo`
Fuzzing	Entradas malformadas que rompen el parser	`atheris`, `AFL++`

La cobertura no mide calidad, mide *alcance*: un 90 % de cobertura con aserciones triviales no prueba nada. Se exige cobertura **más** aserciones significativas.

3. Laboratorio guiado

> Entorno: la aplicación web del proyecto (portal del SOC de xest) o, si aún no está, el repositorio de laboratorio que facilita el docente. Python 3.12 con `pytest` y `coverage`.

Parte A — Lectura comparada (RA1.a, RA1.b, RA1.c)

1. Se entregan tres implementaciones equivalentes de un mismo validador de entrada (en Python, Java y C).
2. Para cada una, localiza: punto de entrada, sink y validación. Señala **dónde puede fallar** cada modelo de ejecución (p. ej., desbordamiento en C, deserialización en Java, `eval` en Python).
3. Rellena la tabla del entregable `PPS-E1`.

Parte B — Suite de pruebas y cobertura (RA1.d)

```
# Instalar dependencias de prueba
python3 -m pip install pytest coverage

# Ejecutar la suite midiendo cobertura
python3 -m coverage run -m pytest -v
python3 -m coverage report -m
python3 -m coverage html   # informe navegable en htmlcov/index.html
```

1. Añade al menos **tres pruebas unitarias** nuevas a una función del proyecto que hoy no esté cubierta.
2. Escribe **una prueba de regresión** que reproduzca un fallo conocido (cadena vacía, número negativo, entrada no ASCII) y verifica que falla antes de arreglar la función y pasa después.

Parte C — Fuzzing básico (refuerzo)

```
python3 -m pip install atheris
# Ejemplo de arnés de fuzzing sobre un parser
python3 fuzz_parser.py -atheris_runs=100000
```

1. Si el fuzzer encuentra una entrada que provoca una excepción no controlada, documéntala como caso de prueba de regresión.

4. Entregables

- **`PPS-E1`** (informe, rúbrica R2): tabla comparada de los tres lenguajes desde la perspectiva de seguridad, con el fallo característico de cada modelo de ejecución identificado sobre el código real.
- **`PPS-E2`** (laboratorio, rúbrica R1): suite ejecutable con las nuevas pruebas y el informe de cobertura (`coverage report` y captura del HTML), incluida la prueba de regresión que demuestra el ciclo fallo→corrección→prueba.

5. Criterios de superación

- El informe distingue correctamente entrada, sink y flujo en las tres implementaciones.
- La suite **ejecuta** (`pytest` en verde) y la prueba de regresión demuestra que cierra el fallo.
- La cobertura mejora respecto a la línea base y las aserciones son significativas, no triviales.

6. Errores frecuentes

- Confundir cobertura alta con código seguro. Cobertura es alcance de la prueba, no ausencia de vulnerabilidades.
- Escribir la prueba de regresión *después* de arreglar el fallo sin verificar que fallaba antes: entonces no prueba nada.
- Analizar el código buscando "funciones peligrosas" en vez de seguir el flujo entrada→sink.

7. Referencias

- OWASP Code Review Guide.
- NIST SP 800-218 (SSDF), práctica PW.7 (revisión de código) y PW.8 (pruebas).
- `pytest` y `coverage.py` — documentación oficial.

PPS · UT2 — Seguridad del lenguaje y del entorno de ejecución

5 horas · RA1 (e) · Servicio SOC: Prevención — configuración segura · Evidencia PPS-E3

Qué vamos a conseguir

Elegir un lenguaje y demostrar cómo limita el entorno una operación.

Antes de empezar

Herramientas: C, Go y Python; Docker y strace en Linux.

Consulta la guía de instalación, la guía de herramientas y las precisiones del curso. Prepara una carpeta propia para resultados; conserva intactos los datos de partida.

Ruta de trabajo

1. Compara los programas de memoria-segura y registra qué protección aporta cada lenguaje y qué riesgo queda en sus dependencias.
2. Resuelve cadena-suministro/FICHA-alumno.md como análisis documental. No instales los nombres sospechosos del fichero de ejemplo.
3. Completa el dictamen: amenaza, criterio, alternativa, decisión y coste operativo. No justifiques una elección solo por popularidad.
4. Abre seccomp/FICHA-seccomp.md. Usa exclusivamente los contenedores indicados y conserva el comando de referencia sin perfil.
5. Repite la misma operación con el perfil aplicado. Compara usuario, capacidades e imagen: deben mantenerse iguales para atribuir el cambio a seccomp.
6. Anota syscall, error y efecto. Cierra PPS-E3 con una prueba de bloqueo y una explicación de los límites del contenedor.

Cómo comprobar que está hecho

Misma operación y privilegios en ambos ensayos; bloqueo atribuible al perfil; dictamen trazable.

> Ejecutar mount sin argumentos solo lista montajes; no demuestra el bloqueo de mount(2).

Desarrollo y fichas

- Explicación completa de la unidad.
- FICHA-alumno
- PLANTILLA-dictamen
- README
- FICHA-seccomp

Entrega y revisión

Los nombres y formatos están en el cuaderno de entregables. Antes de entregar, comprueba alcance, pasos reproducibles, resultados, interpretación y límites. Identifica tu trabajo por número de lista, módulo y evidencia.

Incluye una dificultad que hayas resuelto y explica por qué la solución funciona. Si utilizas una salida de ejemplo, declárala; no la presentes como ejecución propia.

PPS · UT2 — Seguridad del lenguaje y del entorno de ejecución

5 horas · RA1 (e) · Servicio SOC: Prevención — configuración segura · Evidencia PPS-E3

---

1. Objetivo de la unidad

Elegir lenguaje y entorno de ejecución es una decisión de seguridad, no solo de rendimiento o de gusto del equipo. Esta unidad enseña a **razonar esa elección** y a aislar un proceso para que, aunque el código falle, el daño quede contenido.

2. Contenido

2.1 Garantías que aporta (o no) el lenguaje

- **Seguridad de memoria**: Rust, Go, Java, C#, Python la garantizan; C y C++ no. Un servicio expuesto a Internet escrito en C carga con toda la familia de desbordamientos y uso-después-de-liberar salvo mitigación explícita.

- **Tipado**: el tipado fuerte y estático detecta en compilación clases enteras de error; el dinámico las traslada a tiempo de ejecución.
- **Gestión de excepciones**: un lenguaje que obliga a tratar el error (Go con valores de retorno, Rust con `Result`) reduce el fallo silencioso.
- **Cadena de suministro del propio lenguaje**: el intérprete, el compilador y el gestor de paquetes también son superficie (typosquatting en PyPI/npm, compilador troyanizado).

2.2 Entornos de ejecución aislados

Mecanismo	Qué aísla	Uso típico
Sandbox del navegador	JavaScript del cliente respecto al SO	Aplicaciones web
JVM / CLR	Bytecode respecto al nativo	Java / .NET
Contenedor	Proceso respecto al host (namespaces, cgroups)	Despliegue en producción
seccomp	Llamadas al sistema permitidas al proceso	Endurecer contenedores
AppArmor / SELinux	Acceso del proceso a ficheros y capacidades	Bastionado del host

Aislar es reducir lo que un proceso comprometido puede hacer. Un contenedor sin `seccomp` ni usuario no-root comparte demasiada superficie con el host.

3. Laboratorio guiado

Parte A — Dictamen de elección de lenguaje (RA1.e)

1. Escenario: servicio HTTP expuesto a Internet que procesa ficheros subidos por el usuario.
2. Evalúa **tres lenguajes candidatos** (p. ej. Go, Python, C) con estos criterios: seguridad de memoria, madurez del ecosistema de seguridad, facilidad de despliegue aislado, superficie de la cadena de suministro.
3. Emite una recomendación **justificada con los criterios**, no con preferencias.

Parte B — Aislar un proceso con seccomp (RA1.e, refuerzo)

```
# Ver las llamadas al sistema que hace un contenedor
docker run --rm --security-opt seccomp=unconfined alpine \
  sh -c 'strace -f -c whoami' 2>&1 | tail -20

# Aplicar un perfil seccomp restrictivo y comprobar que bloquea
docker run --rm --security-opt seccomp=./perfil-restrictivo.json alpine whoami
```

```
# Ejecutar como usuario no-root y sin capacidades extra
docker run --rm --user 1000:1000 --cap-drop ALL alpine id
```

1. Construye un perfil `seccomp` que permita solo las llamadas necesarias y demuestra empíricamente que una llamada prohibida (p. ej. `mount`) es rechazada.
2. Documenta la diferencia de superficie entre el contenedor por defecto y el endurecido.

4. Entregable

- **`PPS-E3`** (informe, rúbrica R2): dictamen técnico de elección de lenguaje y entorno de ejecución con criterios de seguridad explícitos, más la evidencia del aislamiento con `seccomp` (perfil aplicado y llamada bloqueada).

5. Criterios de superación

- La recomendación se sostiene sobre criterios de seguridad, no sobre familiaridad del equipo.
- El aislamiento se demuestra **empíricamente**: se ve la llamada bloqueada, no se afirma que lo estaría.

6. Errores frecuentes

- Elegir el lenguaje "que ya conoce el equipo" sin evaluar la superficie que introduce.
- Confiar en la ofuscación de bytecode como control de seguridad.
- Dar por aislado un contenedor que corre como root con todas las capacidades.

7. Referencias

- NIST SP 800-190 — Application Container Security Guide.
- OWASP Docker Security Cheat Sheet.
- Documentación de `seccomp`, AppArmor y las opciones `--security-opt` de Docker.

Ejercicio · La cadena de suministro del propio lenguaje (PPS · UT2)

> El intérprete y su gestor de paquetes también son superficie de ataque. Un `pip install` de un nombre mal escrito puede traer código malicioso a producción. Aquí lo cazamos.

Contexto

Alguien ha propuesto este `requirements.txt` para el portal SOC de xest. Antes de que entre en el repositorio, es tu trabajo revisarlo. Fichero: `requirements-sospechoso.txt`.

Qué es un typosquat

Un paquete malicioso publicado con un nombre **casi idéntico** a uno legítimo y popular (`colorama` → `colourama`, `requests` → `reqeusts`), a la espera de que alguien se equivoque al teclear. Al instalarse ejecuta código (en el `setup.py` o al importarse). Campañas reales de este tipo han afectado a PyPI y npm repetidamente.

Tarea

1. Recorre las 12 líneas de `requirements-sospechoso.txt`.
2. Marca cada dependencia como **legítima** o **sospechosa (typosquat / nombre no oficial)**. Para cada sospechosa, escribe **cuál es el nombre correcto** al que se parece y **por qué lo consideras sospechoso** (distancia de edición pequeña, nombre que no existe en PyPI, colisión con la biblioteca estándar…).
3. Escribe el `requirements-corregido.txt` con solo las dependencias legítimas y los nombres bien escritos.

Pista metodológica

No lo hagas "a ojo" y ya está: **contrasta con una fuente**. Compara cada nombre con una lista de paquetes legítimos conocidos. La comparación por parecido la puede hacer una función de la biblioteca estándar de Python (`difflib.get_close_matches`): si un nombre se parece mucho a uno oficial **pero no es exactamente ese**, enciende la alarma. Ese es el patrón que automatizará la CI en la UT7 (análisis de composición de software, SCA).

Entrega parcial

Tu tabla `legítima / sospechosa / nombre correcto / motivo` alimenta el apartado de **cadena de suministro** del dictamen PPS-E3. No es un entregable independiente: es munición para el informe.

> Regla del módulo: no basta con *sospechar*; hay que **verificar contra una segunda fuente**. Un cero sin verificar no vale, y un "parece legítimo" tampoco.

PPS-E3 · Dictamen de elección de lenguaje y entorno + evidencia de aislamiento

> Plantilla del entregable. Rúbrica **R2** (informe técnico). Entrega como `NN-PPS-E3-dictamen.md` (NN = tu número de lista) **más la captura** de la syscall bloqueada por seccomp. Rellena TODOS los apartados. Un dictamen sin criterios explícitos no supera el nivel 2.

---

1. Escenario

Servicio HTTP expuesto a Internet que **procesa ficheros subidos por el usuario** (por ejemplo, recibe adjuntos, los parsea y extrae metadatos). Elige el lenguaje y el entorno de ejecución para ese servicio entre **Go, Python y C**.

2. Criterios de evaluación (con peso)

Justifica el peso que das a cada criterio para ESTE escenario (procesar ficheros no confiables inclina el peso hacia la seguridad de memoria y la superficie de dependencias).

Criterio	Peso (%)	Por qué pesa lo que pesa aquí
Seguridad de memoria		
Madurez del ecosistema de seguridad		
Facilidad de despliegue aislado		
Superficie de la cadena de suministro		
(opcional) Rendimiento / otros		
Total	**100**	

3. Matriz de decisión

Puntúa cada lenguaje en cada criterio (1 = malo … 5 = excelente) y multiplica por el peso.

Criterio (peso)	Go	Python	C
Seguridad de memoria (%)			
Ecosistema de seguridad (%)			
Despliegue aislado (%)			
Cadena de suministro (%)			
Puntuación ponderada			

4. Recomendación

Un párrafo. Nombra el lenguaje elegido y **por qué gana según los criterios**, no por familiaridad del equipo. Menciona explícitamente el criterio que ha sido decisivo.

5. Evidencia de aislamiento (seccomp)

Adjunta:

1. El **comando** con el que aplicaste el perfil `../perfil-restrictivo.json`.
2. La **salida** que demuestra que una syscall prohibida (`mount` vía `mount(2)`, o `ptrace`) es **rechazada** con `Operation not permitted`.
3. Una frase: **qué superficie** reduce ese perfil respecto al contenedor por defecto.

> `(pega aquí tu comando y su salida — la captura de la syscall bloqueada)`

6. Conclusión

Dos líneas: la elección de lenguaje contiene el daño *antes* de que ocurra (menos superficie); el aislamiento lo contiene *cuando* ocurre (menos de lo que un proceso comprometido puede hacer).

Demo · Seguridad de memoria en tres lenguajes (PPS · UT2)

> El mismo problema —copiar un dato de entrada a un espacio de tamaño fijo— en C, Go y Python. Objetivo: **ver** que el lenguaje decide si un dato demasiado largo corrompe la memoria o no. Solo laboratorio aislado de xest. Las opciones que desactivan protecciones (`-fno-stack-protector -z execstack`) NO se usan jamás fuera de una demostración didáctica.

C — sin seguridad de memoria (el riesgo vive en el binario)

```
gcc -fno-stack-protector -z execstack -O0 -o copia copia.c
./copia ana
./copia $(python3 -c 'print("A"*64)')
```

Salida esperada:

```
# entrada corta -> OK
Nombre guardado: ana
Fin del programa.

# entrada de 64 bytes en un buffer de 16 -> se machaca la pila
Segmentation fault (core dumped)
```

Como hemos compilado con `-fno-stack-protector`, el programa **no** avisa con "stack smashing detected" (ese mensaje lo emite justamente el protector que hemos desactivado): la corrupción pasa desapercibida y el proceso muere con un **fallo de segmentación** (código de salida 139). Según cómo caiga la pila, puede que ni siquiera imprima `Nombre guardado:`; lo constante es que **`Fin del programa.` nunca se imprime** — el flujo de control se ha perdido. Ahí es donde, en un servicio real, el atacante coloca su código.

> Comprueba el código de salida: `./copia $(python3 -c 'print("A"*64)'); echo $?` → **139** (128 + señal 11, SIGSEGV). Si recompilas **con** protector (quita `-fno-stack-protector`), verás en su lugar `*** stack smashing detected ***` y `Aborted`: el protector lo detecta y aborta ordenadamente. Ese contraste enseña para qué sirve el protector.

Go y Python — con seguridad de memoria (la corrupción no ocurre)

```
go build -o copia_go copia.go && ./copia_go $(python3 -c 'print("A"*64)')
python3 copia.py $(python3 -c 'print("A"*64)')
```

Salida esperada (ambos):

```
Nombre guardado: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Fin del programa.
```

La entrada larga se imprime entera y el programa **termina normalmente**: no hay buffer fijo que desbordar. La cadena lleva su longitud y el runtime comprueba los accesos.

La idea que se lleva el alumnado

Lenguaje	¿Seguridad de memoria?	Qué pasa con la entrada larga
C / C++	**No** (salvo mitigación explícita)	Corrompe la pila → posible ejecución de código
Go, Rust, Java, C#, Python	**Sí**, de serie	La entrada se maneja sin corromper nada

El riesgo de memoria no desaparece porque "programemos con cuidado": desaparece **por diseño del lenguaje**. Elegir C para un servicio expuesto a Internet es asumir toda esa familia de fallos salvo que se mitigue una a una. Esa decisión es la que se documenta en el dictamen PPS-E3.

Reto · Construir tu propio perfil seccomp (PPS · UT2)

> Solo en la VLAN aislada de xest. Aquí pasas de *usar* un perfil a *escribir* uno, y de la lógica "bloqueo lo peligroso" (deny-list) a la lógica de producción "solo permito lo necesario" (allowlist, deny-by-default). Esto es la segunda mitad de la evidencia PPS-E3.

Dos filosofías de perfil

	`perfil-restrictivo.json` (deny-list)	`perfil-minimo.json` (allowlist)
`defaultAction`	`SCMP_ACT_ALLOW` (permite por defecto)	`SCMP_ACT_ERRNO` (deniega por defecto)
Qué declaras	Las syscalls **peligrosas** que bloqueas	Las syscalls **necesarias** que permites
Riesgo	Se te olvida bloquear alguna	El binario no arranca si falta una
Uso	Didáctico / rápido	**Producción** (así es el perfil por defecto de Docker)

Tarea

1. Parte de una allowlist **deliberadamente incompleta** (por ejemplo, solo `execve`, `write`, `exit_group`). Aplícala a `id` y observa el fallo:

```
bash docker run --rm --security-opt seccomp=./mi-perfil.json alpine id
```

1. **Lee el error.** El kernel mata el proceso al llegar a la primera syscall no permitida. Con `SCMP_ACT_ERRNO` el binario recibe `EPERM` y suele abortar con un mensaje. Para ver exactamente **qué** syscall falta, ejecútalo bajo `strace` con el perfil `unconfined`:

```
bash docker run --rm --security-opt seccomp=unconfined alpine \ sh -c 'apk add -q strace; strace -f id 2>&1 | tail -30'
```

Esa traza es la lista de syscalls reales que `id` necesita.

1. **Itera**: añade a tu allowlist las syscalls que la traza muestra hasta que `id` corra sin error. Ese ciclo — arrancar, leer la syscall que falta, añadirla— es exactamente cómo se endurece un contenedor en producción.

2. **Verifica que sigue conteniendo**: comprueba que, con tu perfil ya funcional para `id`, una syscall peligrosa que NO añadiste (p. ej. `mount`) sigue rechazada.

Qué entregar (parte B de PPS-E3)

- Tu `mi-perfil.json` (allowlist, deny-by-default) con el que `id` corre.
- La **captura** de la syscall bloqueada (el `Operation not permitted` de `mount` o `ptrace`).
- Una frase sobre la **diferencia de superficie** entre el contenedor por defecto y el tuyo.

> No basta con que `id` funcione: eso solo prueba que la allowlist es **suficiente**. Hay que demostrar también que es **restrictiva** — que algo prohibido se rechaza de verdad. Afirmar el aislamiento sin verlo bloquear no acredita el criterio (rúbrica R2).

PPS · UT3 — Nivel de verificación de seguridad: OWASP ASVS y superficie de riesgo

8 horas · RA2 (a, b, c, d) · Servicio SOC: Prevención — gestión de la exposición · Evidencias PPS-E4, PPS-E5

Qué vamos a conseguir

Convertir riesgos en requisitos verificables y modelar amenazas.

Antes de empezar

Herramientas: Navegador, Python, portal local; ASVS y STRIDE.

Consulta la guía de instalación, la guía de herramientas y las precisiones del curso. Prepara una carpeta propia para resultados; conserva intactos los datos de partida.

Ruta de trabajo

1. Lee apps-escenario.md y asigna nivel de verificación según datos, exposición y consecuencias.
2. Registra la versión de ASVS que utiliza tu lista. No mezcles identificadores entre ediciones.
3. Arranca el portal y ejecuta verifica_asvs.py. Lee su código para explicar qué comprueba cada sonda.
4. Clasifica los resultados: verificado, incumplido o no cubierto. Una sonda de cabeceras no verifica todo ASVS.
5. Dibuja procesos, almacenes, flujos y límites de confianza con dfd-portal-soc.md.
6. Completa stride-plantilla.md: amenaza, condición necesaria, impacto, control y prueba de que el control funciona.

Cómo comprobar que está hecho

Versión ASVS indicada; límites de confianza explícitos; cada control tiene una prueba.

> ASVS define requisitos; STRIDE ayuda a plantear amenazas. Un escaneo no sustituye a ninguno.

Desarrollo y fichas

- Explicación completa de la unidad.
- apps-escenario
- dfd-portal-soc
- stride-plantilla

Entrega y revisión

Los nombres y formatos están en el cuaderno de entregables. Antes de entregar, comprueba alcance, pasos reproducibles, resultados, interpretación y límites. Identifica tu trabajo por número de lista, módulo y evidencia.

Incluye una dificultad que hayas resuelto y explica por qué la solución funciona. Si utilizas una salida de ejemplo, declárala; no la presentes como ejecución propia.

PPS · UT3 — Nivel de verificación de seguridad: OWASP ASVS y superficie de riesgo

8 horas · RA2 (a, b, c, d) · Servicio SOC: Prevención — gestión de la exposición · Evidencias PPS-E4, PPS-E5

---

1. Objetivo de la unidad

Antes de corregir vulnerabilidades hay que decidir **cuánta seguridad necesita** la aplicación. No todo servicio requiere el mismo rigor: una web informativa no se verifica como un sistema de pago. Esta unidad enseña a asignar un nivel de verificación con un estándar reconocido (OWASP ASVS) y a modelar de dónde vendrá el ataque (STRIDE), de modo que el esfuerzo posterior sea proporcional al riesgo.

2. Contenido

2.1 OWASP ASVS — Application Security Verification Standard

ASVS define **qué** hay que verificar en una aplicación web, organizado en tres niveles:

Nivel	Para qué aplicación	Ejemplo
Nivel 1	Toda aplicación; verificable desde fuera sin acceso al código	Web pública sin datos sensibles

Nivel	Para qué aplicación	Ejemplo
Nivel 2	Aplicaciones con datos sensibles; la mayoría de las de negocio	Portal con cuentas de usuario, el SOC de xest
Nivel 3	Aplicaciones críticas (salud, finanzas, vidas)	Sistema de pago, historia clínica

Cada nivel es un conjunto de **requisitos de verificación** numerados (V1 autenticación, V2 gestión de sesión, V3 control de acceso, V5 validación, etc.). El nivel asignado determina qué requisitos son de obligado cumplimiento.

2.2 OWASP Top Ten como catálogo de riesgo

El Top Ten (web y móvil) no es un estándar de verificación sino un **mapa de las categorías de riesgo más frecuentes**: control de acceso roto, fallos criptográficos, inyección, diseño inseguro, mala configuración, componentes vulnerables, fallos de identificación y autenticación, etc. Sirve para no dejarse ninguna familia fuera del modelado.

2.3 Modelado de amenazas con STRIDE

STRIDE clasifica las amenazas por lo que atacan:

Letra	Amenaza	Propiedad que rompe
S	Spoofing (suplantación)	Autenticidad
T	Tampering (manipulación)	Integridad
R	Repudiation (repudio)	No repudio
I	Information disclosure (fuga)	Confidencialidad
D	Denial of service	Disponibilidad
E	Elevation of privilege	Autorización

Se aplica sobre un **diagrama de flujo de datos**: por cada elemento (proceso, almacén, flujo, frontera de confianza) se pregunta qué amenaza STRIDE aplica y qué control lo mitiga.

3. Laboratorio guiado

> Entorno: portal del SOC de xest. Referencia: OWASP ASVS 4.0.3 (hoja de requisitos en `PPS/material/plantillas/checklist_asvs.md`).

Parte A — Asignación de nivel ASVS (RA2.a, RA2.b, RA2.c)

1. Se presentan tres aplicaciones de perfil distinto (web informativa, portal con cuentas, pasarela de pago).
2. Asigna a cada una su nivel ASVS y **justifica** por qué, en función de los datos que trata y el impacto de un compromiso.
3. Para el portal del SOC (nivel 2), extrae de la hoja ASVS la **lista de requisitos aplicables** y trázala a las pruebas que los verificarán.

Parte B — Modelado STRIDE (RA2.d)

1. Dibuja el diagrama de flujo de datos del portal del SOC: navegador, servidor de aplicación, base de datos, servicio de autenticación, y las fronteras de confianza entre ellos.
2. Por cada elemento y flujo, enumera las amenazas STRIDE que aplican y el control que las mitiga.
3. Prioriza las amenazas por impacto y probabilidad (matriz de riesgo).

4. Entregables

- **`PPS-E4`** (informe, rúbrica R2): dictamen de nivel ASVS de la aplicación del proyecto y catálogo de requisitos de verificación aplicables, trazados a pruebas.
- **`PPS-E5`** (laboratorio, rúbrica R1): modelo de amenazas STRIDE con diagrama de flujo de datos y riesgos priorizados.

5. Criterios de superación

- El nivel ASVS asignado se justifica por el dato y el impacto, no por defecto.
- El modelo STRIDE cubre todos los elementos del diagrama y cada amenaza tiene un control asociado.
- Los requisitos ASVS son trazables a las pruebas que los comprobarán en las unidades siguientes.

6. Errores frecuentes

- Asignar nivel 3 "por si acaso" a todo: infla el coste y diluye el foco.
- Modelar STRIDE sin diagrama de flujo: se olvidan las fronteras de confianza, que es donde está el riesgo.
- Confundir el Top Ten (categorías de riesgo) con ASVS (requisitos de verificación).

7. Referencias

- OWASP **ASVS 4.0.3** — owasp.org/www-project-application-security-verification-standard
- OWASP **Top Ten** web y móvil.
- Microsoft **STRIDE** y OWASP Threat Modeling Cheat Sheet.

Laboratorio UT3 · Parte A — Tres aplicaciones, tres niveles ASVS (alumnado)

> Material de manos para la asignación de nivel ASVS (evidencia **PPS-E4**). Hoja de requisitos: `PPS/material/plantillas/checklist_asvs.md` (ASVS 4.0.3, nivel 2). Referencia de la unidad: `PPS/unidades/UT3-asvs-superficie-riesgo.md`.

Se te presentan tres aplicaciones reales del ecosistema de xest. Para **cada una**:

1. Asigna un **nivel de verificación ASVS** (L1, L2 o L3).
2. **Justifícalo** con dos argumentos: qué **datos** trata y cuál es el **impacto** de un compromiso. No vale "por si acaso": el nivel debe ser **proporcional al riesgo**.
3. Solo para la **App B** (portal SOC, tu objetivo del módulo): extrae de la hoja ASVS la **lista de requisitos aplicables** y traza cada uno a la **prueba** que lo verificará (una prueba automática, una captura de auditoría o un entregable posterior PPS-E4…E9).

Entrega: `NN-PPS-E4-asvs.md` (dictamen + tabla de requisitos trazados).

App A — `xest.edu` · Web institucional del centro

- **Qué es**: sitio informativo público del CIPFP: noticias, oferta formativa, calendario, formularios de contacto que envían un correo. Sin cuentas de usuario, sin panel privado.
- **Datos que trata**: contenido público + el nombre y correo de quien rellena el formulario de contacto (dato personal básico, no categoría especial).
- **Autenticación**: ninguna de cara al visitante. Solo el gestor de contenidos interno.
- **Impacto de un compromiso**: desfiguración de la portada, spam desde el formulario. No hay datos sensibles que exfiltrar ni dinero en juego.
- **Exposición**: publicada en Internet.

App B — `portal-soc.xest.local` · Portal del SOC de xest *(tu objetivo del módulo)*

- **Qué es**: portal con **cuentas de usuario** (analistas del SOC) que da acceso a paneles de incidentes, alertas de Wazuh y casos de TheHive. Es el servicio que construyes y bastionas a lo largo de PPS y que HE audita e IC vigila.
- **Datos que trata**: credenciales de analistas, tokens de sesión, y **datos operativos de seguridad** (alertas, IOC, incidentes) — información sensible de negocio, aunque no categoría especial del art. 9 RGPD.
- **Autenticación**: login usuario/contraseña; distintos roles (analista, responsable).
- **Impacto de un compromiso**: un atacante que entra ve la telemetría de seguridad de la organización, puede borrar evidencias o suplantar a un analista. Compromete el propio órgano que debería detectar el ataque.
- **Exposición**: interno + accesible por VPN.

App C — `pagos.xest.local` · Pasarela de pago de matrículas y tasas

- **Qué es**: componente que cobra matrículas y tasas del alumnado. Procesa tarjetas a través de un proveedor (TPV virtual) y guarda el histórico de transacciones.
- **Datos que trata**: importe, referencia de pago, datos de titular, tokens de tarjeta. Ámbito **PCI-DSS**; dinero real; impacto económico y legal directo.
- **Autenticación**: cuenta del pagador + doble factor en operaciones sensibles.
- **Impacto de un compromiso**: fraude, robo de datos de tarjeta, sanción regulatoria, pérdida de confianza. Un fallo aquí tiene consecuencias financieras y jurídicas.
- **Exposición**: publicada en Internet, integrada con un proveedor de pago externo.

Rejilla para tu dictamen (rellénala)

App	Nivel ASVS	Dato que lo justifica	Impacto que lo justifica
A · Web institucional			
B · Portal SOC			
C · Pasarela de pago			

Requisitos aplicables al portal SOC (App B, nivel 2) — traza cada uno a su prueba

Toma las familias de la hoja `checklist_asvs.md` (V2 auth, V3 sesión, V4 acceso, V5 validación, V7 errores, V9 comunicaciones, V14 configuración) y completa:

Req. ASVS	Qué verifica	Prueba / evidencia que lo comprobará	¿Cuándo (UT)?
5.3.4	Consultas parametrizadas	`tests/test_seguridad.py::test_login_no_inyectable`	UT4 (PPS-E7)
14.4.1	Cabeceras de seguridad	`verifica_asvs.py` (sonda externa)	UT3 hoy / UT5 (PPS-E9)
…	…	…	…

> Pista: una parte de los requisitos de **V9/V14** los puedes comprobar **hoy mismo** desde fuera con `verifica_asvs.py` contra el portal. Los de **V2/V3/V4/V5** exigen la lógica de login que construirás en UT4-UT5: se trazan a las pruebas de regresión de esos entregables.

Laboratorio UT3 · Parte B — Diagrama de flujo de datos del portal SOC (referencia)

> Base para el modelado STRIDE (evidencia **PPS-E5**). Este DFD es el punto de partida: el alumnado lo redibuja (draw.io/excalidraw/papel) y sobre él enumera las amenazas. Referencia: OWASP Threat Modeling; unidad `PPS/unidades/UT3-asvs-superficie-riesgo.md`.

Convención de un DFD para modelado de amenazas

- **Entidad externa** (rectángulo): actor fuera de nuestro control (usuario, atacante).
- **Proceso** (círculo): código que transforma datos (la app, el servicio de auth).
- **Almacén de datos** (dos líneas paralelas): base de datos, ficheros, caché.
- **Flujo de datos** (flecha): por dónde viaja un dato entre elementos.
- **Frontera de confianza** (línea discontinua): dónde cambia el nivel de confianza. **El riesgo vive en las fronteras**: un dato que las cruza debe validarse/autenticarse.

DFD del portal SOC de xest

```
flowchart LR
    subgraph EXT["Zona no confiable (Internet / VPN)"]
        A["Analista SOC<br/>(navegador)"]
    end

    subgraph DMZ["Frontera 1 · perímetro web"]
        P["Portal SOC<br/>(app web / nginx+backend)"]
    end

    subgraph INT["Zona confiable (backend)"]
        AUTH["Servicio de<br/>autenticación"]
        DB[("Base de datos<br/>usuarios · sesiones · incidentes")]
    end

    A -- "1. HTTPS: usuario/contraseña" --> P
    P -- "2. verifica credenciales" --> AUTH
    AUTH -- "3. ok/ko + rol" --> P
    P -- "4. consulta datos (parametrizada)" --> DB
    DB -- "5. incidentes/alertas" --> P
    P -- "6. HTML + cookie de sesión" --> A

    %% Fronteras de confianza:
    %% F1 Internet<->Portal (flujos 1 y 6)
    %% F2 Portal<->Auth   (flujos 2 y 3)
    %% F3 Portal<->BD     (flujos 4 y 5)
```

Inventario de elementos (lo que STRIDE debe recorrer, sin dejarse ninguno)

#	Elemento	Tipo	Frontera que toca
E1	Analista SOC (navegador)	Entidad externa	F1
P1	Portal SOC (app web)	Proceso	F1, F2, F3
P2	Servicio de autenticación	Proceso	F2
D1	Base de datos (usuarios, sesiones, incidentes)	Almacén	F3
F1	Internet/VPN ↔ Portal	Frontera de confianza	—
F2	Portal ↔ Auth	Frontera de confianza	—
F3	Portal ↔ Base de datos	Frontera de confianza	—

> Regla del ejercicio: **cada proceso, almacén, flujo y frontera** debe aparecer en la tabla STRIDE. Un elemento sin fila es un elemento sin analizar. Y cada amenaza debe llevar su **control mitigador**: una amenaza sin control está *nombrada*, no *modelada*.

Laboratorio UT3 · Parte B — Hoja de modelado STRIDE (alumnado)

> Plantilla del entregable **PPS-E5**. Recorre el DFD (`dfd-portal-soc.md`) elemento a elemento. Por cada amenaza STRIDE que aplique, anota **el control que la mitiga** y un **riesgo** = impacto × probabilidad. Entrega: `NN-PPS-E5-stride.md` + diagrama.

Recordatorio STRIDE

Letra	Amenaza	Propiedad que rompe	Pregunta guía
S	Suplantación (spoofing)	Autenticidad	¿Puede alguien hacerse pasar por otro?
T	Manipulación (tampering)	Integridad	¿Se puede alterar el dato en tránsito o en reposo?
R	Repudio (repudiation)	No repudio	¿Puede negar que hizo una acción?
I	Fuga (information disclosure)	Confidencialidad	¿Se puede leer un dato que no debería?
D	Denegación (denial of service)	Disponibilidad	¿Se puede tumbar el servicio?
E	Elevación (elevation of privilege)	Autorización	¿Puede ganar permisos que no le tocan?

Escala para la matriz de riesgo

- **Impacto**: Bajo (1) · Medio (2) · Alto (3).
- **Probabilidad**: Baja (1) · Media (2) · Alta (3).
- **Riesgo** = Impacto × Probabilidad (1-9). Prioriza de mayor a menor.

Tabla STRIDE por elemento (rellénala)

Elem.	Amenaza (STRIDE)	Descripción concreta en el portal	Control mitigador (requisito ASVS)	Imp.	Prob.	Riesgo
F1	S					
F1	T					
F1	I					
P1	T					
P1	E					
P2	S					
P2	D					
D1	I					
D1	R					
…	…					

Matriz de riesgo (coloca cada amenaza por su id)

↑ Impacto \ Prob. →	Baja (1)	Media (2)	Alta (3)
Alto (3)			
Medio (2)			
Bajo (1)			

Plan de trabajo (salida del modelado)

Lista las 3-5 amenazas de mayor riesgo. Cada una se convierte en un **requisito ASVS a verificar** en UT4/UT5. El modelado no es un dibujo: es tu lista de tareas priorizada.

1. 2. 3.

PPS · UT4 — Vulnerabilidades web I: entrada, inyección, sesión y control de acceso

11 horas · RA3 (a, b, c, d) · Servicio SOC: Prevención — gestión de vulnerabilidades · Evidencias PPS-E6, PPS-E7

Qué vamos a conseguir

Reproducir fallos web y corregir su causa con regresiones.

Antes de empezar

Herramientas: Docker, navegador/proxy, Python, pytest; Semgrep como apoyo.

Consulta la guía de instalación, la guía de herramientas y las precisiones del curso. Prepara una carpeta propia para resultados; conserva intactos los datos de partida.

Ruta de trabajo

1. Prepara el laboratorio local y dos cuentas ficticias con permisos distintos. Anota aplicación y versión.
2. Sigue ficha-PPS-E6-explotacion.md. Para cada familia guarda petición, respuesta y efecto observado en el objetivo autorizado.
3. Abre portal/README-alumnado.md y localiza en el código el dato que llega a la operación sensible.
4. Escribe primero un test que falle ante el defecto. Corrige con consultas parametrizadas, escape contextual, gestión de sesión o autorización por objeto según corresponda.
5. Ejecuta tests positivos y negativos: un usuario autorizado debe poder trabajar y otro no debe acceder al mismo objeto.
6. Entrega PPS-E7 con el cambio, regresiones y comparación antes/después; separa la demostración sobre Juice Shop del código del portal que has corregido.

Cómo comprobar que está hecho

Cuatro familias con evidencia; corrección estructural; permisos legítimos conservados.

> Que desaparezca una alerta del escáner no prueba que la vulnerabilidad esté corregida.

Desarrollo y fichas

- Explicación completa de la unidad.
- ficha-PPS-E6-explotacion
- ficha-PPS-E7-correccion
- README-alumnado

Entrega y revisión

Los nombres y formatos están en el cuaderno de entregables. Antes de entregar, comprueba alcance, pasos reproducibles, resultados, interpretación y límites. Identifica tu trabajo por número de lista, módulo y evidencia.

Incluye una dificultad que hayas resuelto y explica por qué la solución funciona. Si utilizas una salida de ejemplo, declárala; no la presentes como ejecución propia.

PPS · UT4 — Vulnerabilidades web I: entrada, inyección, sesión y control de acceso

11 horas · RA3 (a, b, c, d) · Servicio SOC: Prevención — gestión de vulnerabilidades · Evidencias PPS-E6, PPS-E7

---

1. Objetivo de la unidad

Es la unidad de mayor peso del módulo. Aquí se cierra el círculo del análisis: se **explota** cada familia de vulnerabilidad web en un laboratorio, se **corrige** en el código real del proyecto y se **prueba** que la corrección cierra el hueco y no reaparece. El criterio profesional que rige toda la unidad: una vulnerabilidad no está cerrada hasta que existe una prueba de regresión que fallaría si alguien la reintroduce.

2. Contenido

2.1 Validación de entrada (RA3.a)

- **Lista de permitidos** (allowlist) antes que lista de bloqueados: es más seguro definir lo válido que enumerar lo prohibido.

- **Canonicalización** antes de validar: normalizar codificación (URL, Unicode) para que `%2e%2e` y `..` se traten igual.
- **Codificación de salida** según el contexto (HTML, atributo, JavaScript, URL): la validación de entrada no sustituye a la codificación de salida.

2.2 Inyección (RA3.b)

Tipo	Sink	Mitigación correcta
SQLi	Consulta a base de datos	Consultas parametrizadas (nunca concatenar)
Inyección de comandos	Shell del sistema	Evitar la shell; API con argumentos separados
Inyección de plantilla (SSTI)	Motor de plantillas	No renderizar entrada del usuario como plantilla
XSS reflejado / almacenado / DOM	Respuesta HTML / DOM	Codificación de salida + CSP
LDAP / NoSQL	Directorio / base documental	Parametrización específica del motor

La inyección se produce cuando dato y código se mezclan en el mismo canal. La solución estructural es **separarlos** (parametrizar), no "limpiar" el dato.

2.3 Gestión de sesión (RA3.c)

- Cookies de sesión con `HttpOnly`, `Secure`, `SameSite`.
- Rotación del identificador de sesión tras autenticarse (evita fijación de sesión).
- Expiración por inactividad y absoluta.
- Protección **CSRF**: token sincronizador o `SameSite=Strict/Lax` + verificación de origen.

2.4 Control de acceso (RA3.d)

- Control **por rol** (RBAC) y verificación en **cada** petición, en el servidor.
- Fallo de autorización a nivel de objeto (IDOR): que el usuario A no pueda leer el recurso del usuario B cambiando un identificador en la URL.
- Denegar por defecto: sin permiso explícito, no hay acceso.

3. Laboratorio guiado

> Entorno: laboratorio deliberadamente vulnerable (OWASP Juice Shop o DVWA) para explotar, y el repositorio del proyecto para corregir. Semgrep para la revisión de código.

Parte A — Explotar (RA3.a–d)

```
# Levantar OWASP Juice Shop en local
docker run --rm -p 3000:3000 bkimminich/juice-shop
# Navegar a http://localhost:3000
```

1. Reproduce y documenta, con captura y petición, al menos **cuatro familias distintas**: una inyección, un XSS, un fallo de sesión y un fallo de control de acceso (IDOR).

Parte B — Revisar código (RA3.a, RA3.b)

```
python3 -m pip install semgrep
# Reglas de la comunidad orientadas a seguridad
semgrep --config p/owasp-top-ten --sarif -o semgrep.sarif .
semgrep --config p/security-audit .
```

1. Ejecuta Semgrep sobre el proyecto y **confirma manualmente** cada hallazgo: distingue el verdadero positivo del falso. Un hallazgo sin confirmar no es un hallazgo.

Parte C — Corregir y probar (RA3.a–d)

1. Corrige cada vulnerabilidad confirmada en el código del proyecto: - SQLi → consulta parametrizada. - XSS → codificación de salida contextual. - Sesión → atributos de cookie y rotación. - IDOR → verificación de propiedad del objeto.
2. Escribe, por cada corrección, una **prueba de regresión** que falle contra el código vulnerable y pase contra el corregido.

```
python3 -m pytest tests/test_seguridad.py -v
```

4. Entregables

- **`PPS-E6`** (informe, rúbrica R2): informe vulnerabilidad→corrección→prueba para al menos cuatro familias, con evidencia de explotación y del cierre.
- **`PPS-E7`** (reto integrado, rúbrica R3): confirmaciones en el repositorio que cierran los hallazgos, cada una acompañada de su prueba de regresión, revisadas por un compañero.

5. Criterios de superación

- Cada familia se explota **y** se corrige sobre el código real, no solo se describe.
- Toda corrección lleva prueba de regresión que demuestra el ciclo completo.
- Los falsos positivos de Semgrep están identificados y descartados con justificación.

6. Errores frecuentes

- "Limpiar" la entrada en vez de parametrizar el sink: sigue habiendo inyección con la carga adecuada.
- Corregir el XSS validando la entrada y olvidar la codificación de salida.
- Dar por cerrado un IDOR ocultando el botón en la interfaz sin verificar la autorización en el servidor.
- Aceptar los hallazgos de Semgrep sin confirmarlos: se corrige lo que no es y se ignora lo que sí.

7. Referencias

- OWASP **WSTG 4.2** — secciones de validación de entrada, inyección, gestión de sesión y autorización.
- OWASP Cheat Sheets: Injection Prevention, XSS Prevention, Session Management, Access Control, CSRF Prevention.
- Semgrep — reglas `p/owasp-top-ten` y `p/security-audit`.

Ficha PPS-E6 — Explotación web documentada (UT4 · RA3 a-d · I1/R1)

Entrega: `NN-PPS-E6-explotacion.md` (una ficha por familia). Instrumento I1, rúbrica **R1**. **Entorno:** tu Kali, con los objetivos publicados solo en `127.0.0.1`. Objetivos: Juice Shop (`http://juice.xest.local:3000`) y DVWA (`http://dvwa.xest.local:8081`, nivel `low`). Levántalos con `docker compose -f comun/entornos/objetivos/docker-compose.yml up -d` y resuelve los nombres con `comun/entornos/objetivos/hosts-puesto` (ver guía de instalación, apartado 8).

> Salida cero (R1): cualquier acción sobre un sistema fuera del alcance autorizado califica la práctica con 0. Solo Juice Shop / DVWA de la VLAN de aula.

Qué tienes que hacer

Reproduce y **documenta con petición y captura** cuatro familias **distintas**. Cada ficha debe permitir a un compañero repetir el ataque desde cero.

1. Inyección SQL — login (Juice Shop)

- Objetivo: entrar como el primer usuario sin conocer la contraseña.
- Carga a probar en el campo de correo: `' OR 1=1 --`
- Evidencia: la petición `POST /rest/user/login` y la respuesta con el token de sesión.

2. XSS — campo reflejado o almacenado (Juice Shop / DVWA)

- Objetivo: que el navegador **ejecute** tu marca en la página.
- Carga a probar: `<script>alert(document.domain)</script>` (o `<img src=x onerror=alert(1)>`).
- Evidencia: la URL/campo usado y la captura del `alert` disparado.

3. Fallo de gestión de sesión (DVWA / Juice Shop)

- Objetivo: demostrar **una** de estas: cookie de sesión sin `HttpOnly`/`Secure`, o identificador de sesión que **no cambia** tras iniciar sesión (fijación).
- Evidencia: captura del inspector de cookies (atributos) o de las cabeceras `Set-Cookie` antes y después del login mostrando el **mismo** identificador.

4. Control de acceso roto — IDOR (Juice Shop: cesta/pedidos)

- Objetivo: leer un recurso de **otro** usuario cambiando un identificador en la petición.
- Ejemplo: acceder a `api/Baskets/<otro_id>` o a un pedido ajeno alterando el `id` de la URL.
- Evidencia: petición con el `id` ajeno y la respuesta con datos que no son tuyos.

Formato de cada ficha (usa la plantilla del repo)

Plantilla: `~/xest-lab/2026-2027/PPS/material/plantillas/informe-vulnerabilidad.md`. Por familia: **descripción · evidencia de explotación (petición + captura) · impacto**. La corrección y la prueba de regresión van en E7 (esta ficha solo explota y documenta).

Se evalúa (R1, resumen)

- **Ejecución técnica**: las cuatro familias reproducidas de verdad, no descritas.
- **Evidencia y reproducibilidad**: petición + captura que permiten repetir desde cero.
- **Higiene operativa**: todo dentro de la VLAN; nada contra Internet ni la red del centro.

Ficha PPS-E7 — Corrección estructural + regresión (UT4 · RA3 a-d · I3/R3)

Entrega: PR en el repo del proyecto que cierra las cuatro familias en `portal/app.py` + `tests/test_seguridad.py` con una prueba de regresión por familia + revisión de un compañero. Instrumento I3, rúbrica **R3** (reto integrado).

> Regla del módulo: una vulnerabilidad **no está cerrada** hasta que existe una prueba de regresión que **falla** contra el código vulnerable y **pasa** contra el corregido. Si el test no falla contra el `app.py` de partida, no prueba nada: reescríbelo.

Punto de partida

El fichero `portal/app.py` trae las cuatro familias. Antes de corregir, **confirma** cada sink con Semgrep y con lectura manual entrada→sink (la de UT1):

```
python3 -m pip install semgrep
semgrep --config p/owasp-top-ten --config p/security-audit portal/app.py
```

Aviso importante: **Semgrep no encuentra las cuatro**. Marca el XSS reflejado y la cookie insegura, pero **la SQLi del login y el IDOR de `/recurso` los tienes que encontrar tú** leyendo el flujo entrada→sink. Un hallazgo sin confirmar no es un hallazgo, y una familia que la herramienta no marca **sigue estando ahí**.

Corrección — hazla ESTRUCTURAL, no cosmética

Familia	Corrección correcta	Corrección FALSA (no vale)
SQLi	Consulta **parametrizada** (marcadores ?)	«Limpiar» comillas de la entrada
XSS	**Codificar la salida** por contexto + CSP	Validar solo la entrada
Sesión	`HttpOnly`+`Secure`+`SameSite`+expiración y **rotar** el `sid` tras login	Cambiar el nombre de la cookie
IDOR	Verificar el **dueño** del objeto en cada petición; denegar por defecto	Ocultar el enlace en la interfaz

Pruebas de regresión (`tests/test_seguridad.py`)

Una por familia. Cada una debe **fallar** contra el `app.py` de partida y **pasar** tras tu corrección. Guía de qué comprobar:

- **SQLi**: `POST /login` con `usuario=' OR '1'='1' --` debe devolver **401**, no autenticar.
- **XSS**: `GET /buscar?q=<script>...` — el cuerpo **no** contiene `<script>` crudo; sí `<script>`.
- **Sesión**: tras `POST /login`, la cabecera `Set-Cookie` contiene `HttpOnly` y `SameSite`, y el `sid` emitido **no** es el que traía el cliente (rotación).
- **IDOR**: con la sesión de Ana, `GET /recurso/102` (recurso de Bruno) devuelve **403**.

```
cd portal
python3 -m pytest tests/ -v      # objetivo: 4 passed contra tu código corregido
```

Comprobación del ciclo completo (hazla y déjala escrita en el PR): revierte **una** de tus correcciones, corre la suite y enseña que su prueba **se pone en rojo**. Eso demuestra que la prueba blinda de verdad.

Se evalúa (R3, resumen)

- **Solución técnica**: corrección estructural, no parche cosmético; eliges y justificas.
- **Automatización y repetibilidad**: `pytest tests/` reproduce el cierre desde cero.
- **Trabajo en el repositorio**: confirmaciones atómicas (una por familia), revisadas por un par.
- **Integración**: este portal corregido es el objetivo que **HE UT8** auditará.

Portal SOC de xest — código de partida (PPS · UT4)

Este es el **portal que vais a corregir** durante la UT4. Trae, a propósito, cuatro vulnerabilidades reales que primero explotaréis en Juice Shop/DVWA (E6) y luego cerraréis aquí, en el código, con su prueba de regresión (E7).

> Regla del módulo: una vulnerabilidad **no está cerrada** hasta que existe una prueba de regresión que **falla** contra el código vulnerable y **pasa** contra el corregido.

Puesta en marcha (VLAN de aula aislada)

```
python3 -m pip install -r requirements.txt
flask --app app run          # http://127.0.0.1:5000
```

Usuarios de laboratorio (sintéticos): `ana/ana123`, `bruno/bruno123`, `admin/admin123`. Recursos: `101` es de Ana, `102` de Bruno, `103` de administración.

Mapa de las cuatro familias en este código

Familia	Endpoint	Dónde vive el fallo
SQLi	`POST /login`	la consulta se construye concatenando `usuario`/`password`
XSS	`GET /buscar`, `/comentarios`	la entrada se refleja/almacena sin codificar
Sesión	`POST /login`	cookie `sid` sin `HttpOnly/Secure/SameSite` y sin rotación
IDOR	`GET /recurso/<id>`	no comprueba que el recurso sea del usuario

Qué tenéis que entregar

- **E6** (`ficha-PPS-E6-explotacion.md`): las cuatro familias explotadas en el laboratorio, con petición y captura.
- **E7** (`ficha-PPS-E7-correccion.md`): las cuatro corregidas **en este `app.py`** de forma estructural + una prueba de regresión por familia en `tests/test_seguridad.py`.

No despleguéis este portal fuera de la VLAN de aula. Es deliberadamente vulnerable.

PPS · UT5 — Vulnerabilidades web II: credenciales, bastionado del servidor y contramedidas

8 horas · RA3 (e, f, g) · Servicio SOC: Prevención — configuración segura · Evidencias PPS-E8, PPS-E9

Qué vamos a conseguir

Proteger credenciales y contrastar el bastionado del servicio.

Antes de empezar

Herramientas: Python/Argon2id, nginx, curl; testssl.sh, Lynis y WAF en el entorno preparado.

Consulta la guía de instalación, la guía de herramientas y las precisiones del curso. Prepara una carpeta propia para resultados; conserva intactos los datos de partida.

Ruta de trabajo

1. Lee almacen-credenciales/ficha-E8.md y trabaja sobre una copia de usuarios.json.
2. Registra esquema antiguo y nuevo. Verifica que el alta nueva usa el esquema nuevo y que un acceso correcto migra el registro antiguo.
3. Comprueba acceso incorrecto, hash malformado y repetición de acceso. Justifica coste de Argon2id con una medición en tu equipo.
4. Lee bastionado/ficha-E9.md y guarda respuestas HTTP y auditoría inicial.
5. Valida sintaxis de nginx antes de aplicar el cambio. Configura TLS con certificados del laboratorio y conserva acceso al servicio.
6. Repite la misma batería de pruebas; si trabajas con WAF, incluye una petición legítima y otra de ataque para comprobar falsos positivos y eficacia.

Cómo comprobar que está hecho

Migración verificable; nginx acepta la configuración; diferencia antes/después interpretada.

> Lynis revisa el sistema donde se ejecuta; no sustituye una auditoría remota de un sitio web.

Desarrollo y fichas

- Explicación completa de la unidad.
- README
- ficha-E8
- ficha-E9

Entrega y revisión

Los nombres y formatos están en el cuaderno de entregables. Antes de entregar, comprueba alcance, pasos reproducibles, resultados, interpretación y límites. Identifica tu trabajo por número de lista, módulo y evidencia.

Incluye una dificultad que hayas resuelto y explica por qué la solución funciona. Si utilizas una salida de ejemplo, declárala; no la presentes como ejecución propia.

PPS · UT5 — Vulnerabilidades web II: credenciales, bastionado del servidor y contramedidas

8 horas · RA3 (e, f, g) · Servicio SOC: Prevención — configuración segura · Evidencias PPS-E8, PPS-E9

---

1. Objetivo de la unidad

La aplicación puede ser correcta y aun así el servicio ser inseguro por cómo se almacenan las contraseñas, cómo está configurado el servidor web y qué defensas hay contra automatización. Esta unidad cierra RA3 con el bastionado del servidor y las contramedidas, y exige la evidencia de auditoría **antes y después** que hace defendible el trabajo.

2. Contenido

2.1 Almacenamiento de contraseñas (RA3.e)

- Nunca en claro, nunca con hash rápido (MD5, SHA-1, SHA-256 a secas).
- Funciones de derivación con coste ajustable: **Argon2id** (preferida), bcrypt, scrypt.

- Sal única por usuario (la incluyen estas funciones) y parámetros de coste justificados por el hardware.

```
# Ejemplo con argon2-cffi
from argon2 import PasswordHasher
ph = PasswordHasher(time_cost=3, memory_cost=65536, parallelism=4)
hash_ = ph.hash("contraseña-del-usuario")   # almacenar hash_
ph.verify(hash_, "contraseña-del-usuario")  # verificar
```

2.2 Bastionado del servidor web (RA3.f)

- Cabeceras de seguridad: `Strict-Transport-Security`, `Content-Security-Policy`, `X-Content-Type-Options`, `Referrer-Policy`.
- TLS moderno: solo 1.2/1.3, suites fuertes, sin renegociación insegura.
- Ocultar versión, deshabilitar métodos innecesarios (TRACE), permisos mínimos de fichero.
- Referencia: **CCN-STIC 652** (Nginx), **CIS Benchmark** Nginx/Apache.

2.3 Contramedidas anti-automatización (RA3.g)

- **HSTS** para forzar HTTPS; **CSP** para contener XSS.
- Limitación de tasa (rate limiting) contra fuerza bruta y registro masivo.
- CAPTCHA / bloqueo progresivo tras intentos fallidos.
- WAF (ModSecurity + OWASP Core Rule Set) en modo aprendizaje primero, luego bloqueo, ajustando falsos positivos.

3. Laboratorio guiado

Parte A — Migrar el almacén de contraseñas (RA3.e)

1. Sustituye el almacenamiento actual del proyecto por **Argon2id** con parámetros justificados.
2. Implementa la migración transparente: al validar con el esquema antiguo, re-hashea con el nuevo.

Parte B — Bastionar Nginx y auditar antes/después (RA3.f)

```
# Auditoría ANTES
testssl.sh --quiet --color 0 https://staging.local > antes-tls.txt
sudo lynis audit system > antes-lynis.txt

# Aplicar la configuración endurecida (ver PPS/material/baseline/nginx-hardening.conf)
sudo nginx -t && sudo systemctl reload nginx

# Auditoría DESPUÉS
testssl.sh --quiet --color 0 https://staging.local > despues-tls.txt
```

```
sudo lynis audit system > despues-lynis.txt
diff antes-tls.txt despues-tls.txt
```

1. Aplica la baseline y demuestra la mejora con el diff de las auditorías.

Parte C — WAF con OWASP CRS (RA3.f, RA3.g)

```
# ModSecurity + CRS en modo detección primero
# SecRuleEngine DetectionOnly  -> revisar log -> SecRuleEngine On
tail -f /var/log/modsec_audit.log
```

1. Despliega ModSecurity con CRS, provoca un falso positivo real con tráfico legítimo del proyecto y **ajusta la regla** en vez de desactivar el WAF. Documenta el ajuste.

4. Entregables

- **`PPS-E8`** (laboratorio, rúbrica R1): servidor web bastionado con evidencia de auditoría antes y después (testssl.sh + Lynis) y el diff que prueba la mejora.
- **`PPS-E9`** (reto integrado, rúbrica R3): configuración de WAF y contramedidas anti-automatización con justificación de cada regla y del ajuste del falso positivo.

5. Criterios de superación

- Las contraseñas se almacenan con Argon2id y parámetros justificados; hay migración transparente.
- La mejora del bastionado se **demuestra** con auditoría antes/después, no se afirma.
- El falso positivo del WAF se resuelve afinando la regla, no apagando el control.

6. Errores frecuentes

- Subir la iteración de bcrypt "a ojo" sin medir el coste real en el hardware de producción.
- Añadir cabeceras de seguridad y romper la aplicación por una CSP demasiado estricta sin fase de solo-informe.
- Desactivar el WAF al primer falso positivo en vez de ajustar la regla: se pierde el control entero por un caso.

7. Referencias

- OWASP Password Storage Cheat Sheet · `argon2-cffi`.
- **CCN-STIC 652** (Nginx) · CIS Benchmarks Nginx/Apache.
- OWASP ModSecurity **Core Rule Set** · testssl.sh · Lynis.

Laboratorio UT5 — Credenciales, bastionado y contramedidas

Material de manos de la UT5 (RA3.e-g). Dos bloques, uno por sesión de 4 h.

`almacen-credenciales/` — PPS-E8 (Sesión 1, RA3.e)

Fichero	Distribuir	Qué es
`store.py`	Sí	Almacén de partida: SHA-256 sin sal. Punto a migrar.
`usuarios.json`	Sí	Dataset sintético: 10 usuarios con hash SHA-256.
`crack_demo.py`	Sí	Demo del coste SHA-256 vs Argon2id.
`ficha-E8.md`	Sí	Enunciado del alumnado.
`SOLUCION-E8.md`	**No**	Solución + claves del dataset + salidas esperadas.
`solucion/store_argon2.py`	**No**	Almacén migrado (Argon2id + migración transparente).
`solucion/test_migracion.py`	**No**	Suite con el test de regresión.

Comprobación rápida (profesor):

```
cd almacen-credenciales/solucion && cp ../usuarios.json usuarios.json
python3 -m pytest -q        # 6 passed
python3 store_argon2.py ana verano2024   # OK + migra ana a argon2id
```

`bastionado/` — PPS-E9 (Sesión 2, RA3.f-g)

Fichero	Distribuir	Qué es
`auditar.sh`	Sí	Auditoría antes/después (testssl + lynis) + diff.
`docker-compose.waf.yml`	Sí	WAF ModSecurity + OWASP CRS ante el portal.

Fichero	Distribuir	Qué es
`modsec/tuning.conf`	Sí	Ajuste acotado del falso positivo (ejemplo).
`ficha-E9.md`	Sí	Enunciado del alumnado.
`SOLUCION-E9.md`	**No**	Mapa debilidad→contramedida + demos + falso positivo guiado.

Reutiliza (no se duplica aquí): - Config floja de partida: `comun/entornos/objetivos/portal-soc/nginx.conf`. - Baseline endurecido: `proyectos/proyectofinal/PPS/material/baseline/nginx-hardening.conf`.

> Todo el bloque de bastionado se ejecuta en la **VLAN aislada de xest**. Nunca contra Internet ni la red del centro.

Ficha de laboratorio · PPS-E8 — Migración del almacén de contraseñas a Argon2id

UT5 · RA3.e · Rúbrica R1 (práctica de laboratorio) · Instrumento I1

> Entorno: tu equipo del aula (Python 3). No necesita la VLAN de objetivos. Requisito: `pip install argon2-cffi pytest`.

Contexto

El portal SOC de xest guarda las contraseñas de sus usuarios con **SHA-256 sin sal** (`store.py`, `usuarios.json`). Es indefendible: SHA-256 es un hash rápido, sin sal las claves iguales colisionan y sirven las tablas precalculadas. Tu trabajo es migrar el almacén a **Argon2id** sin obligar a nadie a cambiar su contraseña.

Material que se te entrega

Fichero	Qué es
`store.py`	Almacén de partida (SHA-256). El código que vas a mejorar.

Fichero	Qué es
`usuarios.json`	10 usuarios con su hash SHA-256. Tu lote de datos.
`crack_demo.py`	Demo del coste: rompe un PIN contra SHA-256 y contra Argon2id.

Tareas

1. **Mide el problema.** Ejecuta `python3 crack_demo.py` y anota cuánto tarda el ataque contra cada esquema. Esa diferencia es tu justificación técnica.
2. **Migra el alta.** Cambia `crear_usuario` para que use Argon2id con parámetros justificados (`time_cost`, `memory_cost`, `parallelism`). Justifica la elección midiendo cuánto tarda un `hash()` en tu equipo (objetivo: 0,3–0,5 s).
3. **Migración transparente.** Cambia `verificar` para que: - si el hash guardado ya es Argon2id, valide con Argon2id; - si es el hash antiguo (SHA-256) y la clave es correcta, **re-hashee al vuelo** con Argon2id y actualice `usuarios.json`. El usuario no debe notar nada. - Añade `check_needs_rehash` para re-hashear si suben los parámetros.
4. **Prueba de regresión.** Escribe una suite `pytest` que incluya, como mínimo, un test que **fallaría si alguien reintrodujera el hash débil** (que un alta nunca vuelva a escribir un SHA-256 de 64 hex).
5. **Verifica la migración.** Demuestra con una ejecución que un usuario legacy, tras un login correcto, queda guardado como `argon2id` en el fichero.

Qué entregar (`NN-PPS-E8.md` + PR)

- El diff / PR de `store.py` migrado.
- La suite de pruebas y su salida (`pytest -v`) en verde.
- Tabla con los **parámetros elegidos y su justificación** (tiempo medido).
- Evidencia de la migración transparente: `usuarios.json` de un usuario antes (`sha256`) y después (`argon2id`) de un login.

Criterio de superación

Las contraseñas se almacenan con Argon2id y parámetros justificados **medidos, no a ojo**; existe migración transparente; existe una prueba de regresión que caza la reintroducción del hash débil. Sin esa prueba, el trabajo **no** está cerrado.

Ficha de laboratorio · PPS-E9 — Bastionado del servidor con auditoría antes/después

UT5 · RA3.f, RA3.g · Rúbrica R2 (informe técnico) · Instrumento I2

> Entorno: **VLAN aislada de xest**. Objetivos levantados (`~/xest-lab/2026-2027/comun/entornos/objetivos → docker compose up -d`). Herramientas: `testssl.sh`, `lynis`, Docker. Regla de oro: nada fuera del laboratorio, nunca contra Internet ni la red del centro.

Contexto

El portal SOC arranca con una configuración de nginx **deliberadamente floja** (`objetivos/portal-soc/nginx.conf`): sin cabeceras de seguridad, revela la versión, admite TRACE, lista directorios y no limita la tasa. Tu trabajo es bastionarlo y **demostrar** la mejora midiéndola antes y después, no afirmándola.

Material que se te entrega / reutilizas

Recurso	Dónde
Config de partida (floja)	`comun/entornos/objetivos/portal-soc/nginx.conf`
Baseline endurecido de referencia	`proyectos/proyectofinal/PPS/material/baseline/nginx-hardening.conf`
Script de auditoría antes/después	`auditar.sh` (esta carpeta)
WAF ModSecurity + CRS	`docker-compose.waf.yml` + `modsec/tuning.conf`

Tareas

Parte A — Auditar el "antes"

```
./auditar.sh antes https://portal-soc.xest.local
```

Guarda `evidencia/antes-tls.txt` y `evidencia/antes-lynis.txt`. Anota el **Hardening index** de Lynis y las carencias TLS/cabeceras que detecta testssl.

Parte B — Aplicar el baseline y auditar el "después"

1. Sustituye la config floja por el baseline endurecido (adaptando dominio, rutas y certificados). Repasa qué añade cada bloque: HSTS, CSP, `X-Content-Type-Options`, `Referrer-Policy`, TLS 1.2/1.3, `server_tokens off`, bloqueo de TRACE, `limit_req`.

2. Valida y recarga: `bash sudo nginx -t && sudo systemctl reload nginx`
3. Audita el "después" y genera el diff: `bash ./auditar.sh despues https://portal-soc.xest.local evidencia/diff-tls.txt` es tu prueba objetiva de la mejora.

Parte C — WAF con OWASP CRS y ajuste de un falso positivo (RA3.g)

1. Levanta el WAF por delante del portal: `bash docker compose -f docker-compose.waf.yml up -d docker logs -f portal-waf # log de auditoría de ModSecurity` Empieza en `DetectionOnly` (solo registra).
2. Genera tráfico legítimo que dispare un **falso positivo** (por ejemplo, un texto de formulario legítimo que contenga comillas o palabras como `select`) y localízalo en el log (regla que lo marca, parámetro, ruta).
3. **Ajusta la regla** de forma acotada (`modsec/tuning.conf`), no desactives el WAF. Documenta la exclusión y su justificación.
4. Pasa a bloqueo (`MODSEC_RULE_ENGINE=On`) y demuestra dos cosas: el tráfico legítimo ya pasa, y un ataque real (p. ej. `?q=1' OR '1'='1`) **sigue bloqueado**.

Qué entregar (`NN-PPS-E9-bastionado.md`)

- `antes-tls.txt`, `despues-tls.txt`, `antes-lynis.txt`, `despues-lynis.txt` y `diff-tls.txt`.
- Informe R2: resumen ejecutivo + detalle técnico + trazabilidad carencia → cambio de config → evidencia de que se cerró.
- El ajuste del falso positivo del WAF con su justificación y la prueba de que el control sigue activo para ataques reales.
- Cita de la norma aplicable (**CCN-STIC 652**, CIS Benchmark nginx) explicando por qué aplica.

Criterio de superación

La mejora del bastionado se **demuestra con el diff antes/después**, no se afirma. El falso positivo del WAF se resuelve **afinando la regla**, nunca apagando el control. Sin el "antes" guardado, el trabajo no es defendible.

PPS · UT6 — Seguridad en aplicaciones para dispositivos móviles

9 horas · RA4 (a, b, c, d, e) · Servicio SOC: Prevención — superficie móvil · Evidencias `PPS-E10`, `PPS-E11`

Qué vamos a conseguir

Relacionar código móvil, tráfico y decisiones del servidor.

Antes de empezar

Herramientas: jadx/MobSF, Python y mitmproxy; APK o fuentes didácticas incluidas.

Consulta la guía de instalación, la guía de herramientas y las precisiones del curso. Prepara una carpeta propia para resultados; conserva intactos los datos de partida.

Ruta de trabajo

1. Lee README.md y registra si analizas un APK real del laboratorio o el árbol de fuentes proporcionado.
2. Resuelve fichas/E10-analisis-estatico.md: vincula cada hallazgo a permiso, fichero o llamada concreta.
3. Si usas MobSF, conserva versión, hash del APK e informe. Confirma a mano y descarta alertas sin evidencia.
4. Sigue la ficha de tráfico con los clientes y backend locales. Distingue HTTP, TLS con CA de laboratorio y fijación de certificado.
5. Compara el servidor de compras vulnerable y el seguro. Prueba importe, usuario, producto y repetición de la misma operación.
6. Entrega el informe diferenciando análisis de fuentes, simulación y ejecución Android: no presentes uno como prueba del otro.

Cómo comprobar que está hecho

Cada hallazgo tiene ubicación; tráfico interpretado; compra validada en servidor.

> Un árbol de fuentes equivalente sirve para razonar, pero no acredita haber ejecutado un APK.

Desarrollo y fichas

- Explicación completa de la unidad.
- build-apk
- E10-analisis-estatico
- E11-trafico-compras
- LEER

Entrega y revisión

Los nombres y formatos están en el cuaderno de entregables. Antes de entregar, comprueba alcance, pasos reproducibles, resultados, interpretación y límites. Identifica tu trabajo por número de lista, módulo y evidencia.

Incluye una dificultad que hayas resuelto y explica por qué la solución funciona. Si utilizas una salida de ejemplo, declárala; no la presentes como ejecución propia.

PPS · UT6 — Seguridad en aplicaciones para dispositivos móviles

9 horas · RA4 (a, b, c, d, e) · Servicio SOC: Prevención — superficie móvil · Evidencias `PPS-E10`, `PPS-E11`

---

1. Objetivo de la unidad

La aplicación móvil corre en un dispositivo que no controlamos y puede ser desensamblada por su dueño. Esta unidad enseña a analizar sus riesgos propios —permisos, almacenamiento, firma, tráfico, secretos en el binario— con la metodología OWASP MASTG.

2. Contenido

2.1 Modelos de permisos (RA4.a)

- **Android**: permisos declarados en el manifiesto, concedidos en tiempo de ejecución; llamadas al sistema protegidas.
- **iOS**: permisos por capacidad, consentimiento explícito del usuario.

- Principio de mínimo privilegio: una app que pide más permisos de los que necesita es un riesgo y una señal.

2.2 Almacenamiento seguro (RA4.b)

- Datos sensibles en el almacén protegido del sistema: **Android Keystore**, **iOS Keychain**.
- Cifrado de bases de datos locales (SQLCipher) y no dejar secretos en `SharedPreferences`/`plist` en claro.

2.3 Firma y validación de compras (RA4.c)

- Firma de la aplicación: garantiza origen e integridad; verificarla en la distribución.
- **Validación de compras integradas en el servidor**, nunca solo en el cliente: el cliente es manipulable.

2.4 Tráfico y protocolos inseguros (RA4.d)

- Detección de HTTP en claro, TLS mal configurado, ausencia de fijación de certificado.
- Interceptación con proxy para ver qué envía realmente la app.

2.5 Fugas en el binario (RA4.e)

- Claves de API, URLs internas, credenciales incrustadas en el APK/IPA.
- Inspección estática del binario para encontrarlas.

3. Laboratorio guiado

> Entorno: APK de laboratorio (o app propia), MobSF, jadx y mitmproxy. Emulador o dispositivo de laboratorio; alternativa virtualizada documentada para quien no disponga de hardware.

Parte A — Análisis estático con MobSF y jadx (RA4.a, RA4.b, RA4.e)

```
# MobSF en contenedor
docker run --rm -p 8000:8000 opensecurity/mobile-security-framework-mobsf
# Subir el APK por la interfaz web y revisar: permisos, secretos, almacenamiento

# Decompilar para inspección manual
jadx -d salida_jadx app-laboratorio.apk
grep -rniE "api[_-]?key|secret|password|https?://" salida_jadx/sources | head
```

1. Extrae del informe MobSF: permisos excesivos, secretos incrustados y almacenamiento inseguro. Confirma manualmente con jadx.

Parte B — Interceptar el tráfico (RA4.d)

```
# Proxy de interceptación
mitmproxy --mode regular --listen-port 8080
# Configurar el proxy en el dispositivo/emulador e instalar el CA de mitmproxy
```

1. Captura el tráfico de la app y demuestra un protocolo inseguro (HTTP en claro o ausencia de fijación de certificado). Si la app fija el certificado, documenta el intento y el bloqueo.

Parte C — Diseño de validación de compras (RA4.c)

1. Diseña el flujo de validación de una compra integrada **contra servidor**, con firma verificable, y explica por qué la validación solo en cliente es insuficiente.

4. Entregables

- **`PPS-E10`** (informe, rúbrica R2): informe MASTG de la aplicación con hallazgos (permisos, secretos, almacenamiento) y remediación propuesta.
- **`PPS-E11`** (laboratorio, rúbrica R1): captura y análisis del tráfico con demostración del protocolo inseguro o del intento de interceptación bloqueado.

5. Criterios de superación

- Los hallazgos de MobSF se confirman manualmente con jadx.
- La interceptación se demuestra sobre tráfico real de la app.
- El diseño de validación de compras sitúa la comprobación en el servidor.

6. Errores frecuentes

- Confiar en la ofuscación del APK como si fuera cifrado.
- Validar la compra solo en el cliente.
- Guardar el token o la clave en almacenamiento no protegido del dispositivo.

7. Referencias

- OWASP **MASTG** y **MASVS**.
- OWASP Mobile Top Ten.
- MobSF, jadx, mitmproxy — documentación oficial.

Cómo obtener el APK de NotasXest (opcional)

La carpeta `app-laboratorio/` contiene las **fuentes** de la app vulnerable. Para analizarla con MobSF hace falta un `.apk`. Hay dos caminos:

Opción 1 (recomendada en aula) — no compilar

No necesitas el APK para dar la clase:

- **jadx/grep**: trabaja directamente sobre `../salida_jadx/`, que ya imita la salida de `jadx` (mismas fuentes + manifiesto). El `grep` de secretos funciona tal cual.
- **MobSF**: lee el informe pregrabado `../mobsf/informe-mobsf-ejemplo.json`, coherente con estas fuentes.

Opción 2 — compilar el APK (si tienes Android SDK)

Estas fuentes están pensadas para dejarlas caer en un proyecto Android mínimo (`minSdk 23`, `targetSdk 28`) y compilar en depuración:

```
# dentro de un proyecto Android que use estas fuentes y este AndroidManifest.xml
./gradlew assembleDebug
# el APK queda en app/build/outputs/apk/debug/app-debug.apk
```

Copia: - `AndroidManifest.xml` → `app/src/main/AndroidManifest.xml` - `java/com/xest/notaslab/*.java` → `app/src/main/java/com/xest/notaslab/` - `res/xml/network_security_config.xml` → `app/src/main/res/xml/`

Sube el `app-debug.apk` resultante a MobSF. El APK firmado en depuración es suficiente para el laboratorio (de hecho MobSF marcará la firma de depuración como hallazgo).

> El objetivo del ejercicio es **analizar**, no publicar. El APK de laboratorio no se distribuye fuera del aula.

Ficha PPS-E10 · Análisis estático de una app móvil (MobSF + jadx)

UT6 · RA4 a,b,e · Instrumento I1 · Rúbrica R1 · Semana 13 Entorno: VLAN de laboratorio de xest. App de laboratorio `NotasXest` (`app-laboratorio/`). Herramientas: MobSF (contenedor), jadx, `grep`.

> Regla de aula: todo el análisis ocurre sobre el APK/fuentes de laboratorio, en la VLAN aislada. No se analiza ninguna app ajena sin autorización por escrito.

Objetivo

Producir un informe de análisis **estático** de `NotasXest` que documente, con evidencia, tres clases de hallazgo del OWASP MASTG y proponga su remediación:

1. **Permisos excesivos** (mínimo privilegio roto).
2. **Secretos incrustados** en el binario (claves, contraseñas, URLs internas).
3. **Almacenamiento inseguro** de datos sensibles.

Cada hallazgo de MobSF debe **confirmarse a mano** con jadx/`grep`. Un hallazgo que no confirmas, no lo pones.

Pasos

1. **MobSF.** Arranca el contenedor y sube el APK (o, si no hay APK/emulador, lee el informe pregrabado `mobsf/informe-mobsf-ejemplo.json`). `bash docker run --rm -p 8000:8000 opensecurity/mobile-security-framework-mobsf # navegador -> http://localhost:8000 -> subir el APK` Extrae del informe: puntuación, permisos peligrosos, secretos, hallazgos de manifiesto y de almacenamiento.

2. **Confirmar con jadx/grep.** Sobre las fuentes decompiladas (`salida_jadx/sources`, ya provista; o genera la tuya con `jadx -d salida_jadx <apk>`): `bash grep -rniE "api[_-]?key|secret|password|https?://" salida_jadx/sources | head` Localiza el fichero y la línea exacta de cada secreto y del almacenamiento inseguro.

3. **Permisos.** Abre `salida_jadx/resources/AndroidManifest.xml` y lista los permisos `dangerous`. Para cada uno, argumenta si una app de notas lo necesita.

Entregable

`NN-PPS-E10-movil.md` siguiendo la plantilla del repo (`PPS/material/plantillas/informe-vulnerabilidad.md`, adaptada a móvil):

- **Resumen**: tabla de hallazgos (permisos / secretos / almacenamiento) con severidad.
- **Por hallazgo**: descripción, **evidencia** (fichero:línea del grep o captura de MobSF), impacto y **remediación concreta** (p. ej. "mover a Android Keystore", "quitar READ_SMS").
- **Trazabilidad**: cada hallazgo de MobSF apunta a su confirmación manual.

Se evalúa (R1)

Ejecución técnica, **evidencia y reproducibilidad** (comandos y salidas para repetirlo), verificación (confirmaste el hallazgo con una segunda herramienta), higiene operativa (trabajo en la VLAN) y tiempo. La ofuscación **no** cuenta como control: si el secreto sale con jadx, es un secreto público.

Ficha PPS-E11 · Tráfico móvil y validación de compras

UT6 · RA4 c,d · Instrumento I2 · Rúbrica R2 · Semana 13 Entorno: VLAN de laboratorio. Backend `NotasXest` (`trafico/`), mitmproxy, servidores de compras (`compras/`).

> Regla de aula: la interceptación se hace **solo** contra el backend de laboratorio, en la VLAN aislada. Nada de interceptar tráfico ajeno.

Objetivo

1. **Interceptar** el tráfico de la app y **demostrar** un protocolo inseguro: contraseña y clave de API viajando en **HTTP en claro**.
2. **Fijación de certificado (pinning)**: demostrar que una app que pinnea el certificado **no** se puede interceptar aunque se instale el CA de mitmproxy (documenta el bloqueo).
3. **Validación de compras**: argumentar, con una demostración, por qué la validación de una compra integrada **va en el servidor** y nunca solo en el cliente.

Pasos

A · HTTP en claro (RA4.d)

```
# terminal 1: backend en claro
python3 trafico/backend_inseguro.py --host 127.0.0.1 --port 8090
# terminal 2: mitmproxy con el addon que resalta secretos
mitmdump -q -s trafico/mitm_addon.py --mode regular --listen-port 8080
# terminal 3: la "app" hace login + sync a través del proxy
python3 trafico/app_cliente.py --base http://127.0.0.1:8090 --proxy http://127.0.0.1:8080
```

Captura la petición `/login`: se ve la **contraseña** en el cuerpo y la **X-Api-Key** en la cabecera, en claro. Esa captura es tu evidencia.

B · Pinning (RA4.d, parte defensiva)

```
# genera cert de laboratorio y arranca el backend en TLS (ver README del lab)
python3 trafico/backend_inseguro.py --tls        # 127.0.0.1:8443
python3 trafico/app_cliente_pinning.py --calcular-pin        # copia el pin
# conexión directa: pin coincide -> OK
python3 trafico/app_cliente_pinning.py --pin <PIN>
# a través de mitmproxy: pin distinto -> BLOQUEADO
python3 trafico/app_cliente_pinning.py --pin <PIN> --proxy 127.0.0.1:8080
```

Documenta el intento y el bloqueo: eso también es un resultado válido.

C · Validación de compras (RA4.c)

Compara los dos servidores y **razona** la diferencia:

```
cd compras && python3 -m pytest -v
```

El servidor **vulnerable** activa premium con `{"purchase_valid": true}` enviado por el cliente; el **seguro** lo rechaza y exige un recibo firmado verificado en el servidor.

Entregable

`NN-PPS-E11-trafico.md`: - **Captura y análisis** del tráfico (HTTP en claro): qué dato se filtra y por qué. - **Pinning**: evidencia del bloqueo o del intento. - **Compras**: diseño del flujo correcto (recibo firmado → verificación en servidor) y argumento de por qué la validación solo en cliente es insuficiente. Referencia a la salida del `pytest` como evidencia.

Se evalúa (R2)

Corrección técnica, estructura y **trazabilidad** (hallazgo → evidencia → remediación), fundamento metodológico (MASTG/MASVS), priorización del riesgo y redacción.

MobSF en el laboratorio de UT6

Con APK y contenedor (camino normal)

```
docker run --rm -p 8000:8000 opensecurity/mobile-security-framework-mobsf
# navegador -> http://localhost:8000 -> arrastrar el APK de NotasXest
```

Lee, en este orden, las secciones del informe: 1. **Security Score** y grado. 2. **Permissions**: filtra por `dangerous`. 3. **Manifest Analysis**: `debuggable`, `clear text traffic`, `allowBackup`, actividad exportada, `network_security_config`. 4. **Secrets** / **Code Analysis**: clave de API, contraseña incrustada, almacenamiento inseguro (`SharedPreferences`).

Cada hallazgo se **confirma a mano** con jadx/grep (ver ficha PPS-E10).

Sin APK ni emulador (alternativa documentada en la unidad)

Usa `informe-mobsf-ejemplo.json` (este directorio): es un recorte coherente con `app-laboratorio/`, con los mismos permisos, secretos y hallazgos. Sirve para que el alumnado practique la **lectura e interpretación** del informe y la redacción de PPS-E10 aunque no haya dispositivo. La confirmación manual se hace igualmente sobre `salida_jadx/`.

```
python3 -m json.tool informe-mobsf-ejemplo.json | less
```

PPS · UT7 — DevSecOps: control de versiones, construcción e integración continua

10 horas · RA5 (a, b, c, g) · Servicio SOC: Prevención — canalización segura · Evidencias PPS-E12, PPS-E13

Qué vamos a conseguir

Impedir que un cambio inseguro llegue a la rama de entrega.

Antes de empezar

Herramientas: Git, pytest, Semgrep, Trivy, Gitleaks y Checkov; forja/runner del aula.

Consulta la guía de instalación, la guía de herramientas y las precisiones del curso. Prepara una carpeta propia para resultados; conserva intactos los datos de partida.

Ruta de trabajo

1. Crea rama de trabajo en el proyecto local y realiza un cambio pequeño con mensaje descriptivo.
2. Configura la protección en la forja indicada por el profesor: revisión y comprobaciones requeridas. Git local por sí solo no impone revisiones.
3. Ejecuta los controles uno a uno y registra herramienta, versión, reglas y bases utilizadas.
4. Añade un defecto sintético conocido a una rama de prueba. Verifica que el control pertinente devuelve fallo.
5. Integra los controles en la canalización sin ocultar errores; verifica que la fusión queda bloqueada.
6. Corrige, repite la canalización y documenta la revisión. Sin forja disponible, practica controles locales y deja PPS-E12 pendiente de demostración real.

Cómo comprobar que está hecho

Caso inseguro bloqueado; caso corregido aceptado; protección demostrada en servidor.

> Sin Internet, las bases de vulnerabilidades, reglas, acciones y contenedores deben estar precargados.

Desarrollo y fichas

- Explicación completa de la unidad.
- ficha-PPS-E12-rama-protegida
- ficha-PPS-E13-ci-bloquea

Entrega y revisión

Los nombres y formatos están en el cuaderno de entregables. Antes de entregar, comprueba alcance, pasos reproducibles, resultados, interpretación y límites. Identifica tu trabajo por número de lista, módulo y evidencia.

Incluye una dificultad que hayas resuelto y explica por qué la solución funciona. Si utilizas una salida de ejemplo, declárala; no la presentes como ejecución propia.

PPS · UT7 — DevSecOps: control de versiones, construcción e integración continua

10 horas · RA5 (a, b, c, g) · Servicio SOC: Prevención — canalización segura · Evidencias `PPS-E12`, `PPS-E13`

---

1. Objetivo de la unidad

Todo lo aprendido en las unidades anteriores se automatiza aquí: la canalización de integración continua ejecuta los controles de seguridad en cada cambio y **bloquea** el que no cumple. El criterio de la unidad es que la canalización impida el error, no que lo avise cuando ya está en producción.

2. Contenido

2.1 DevOps y DevSecOps (RA5.a)

- Principios: flujo (entregas pequeñas y frecuentes), retroalimentación rápida, experimentación continua.
- DevSecOps = mover la seguridad a la izquierda: los controles se ejecutan en el desarrollo, no en una auditoría final.

2.2 Control de versiones (RA5.b)

- Estrategia de ramas y **protección de la rama principal**: nada entra sin revisión y sin CI en verde.
- Revisión por pares obligatoria; roles y permisos mínimos.
- Firma de confirmaciones (opcional pero recomendable) para trazabilidad de autoría.

2.3 Construcción e integración continua (RA5.c)

Controles que ejecuta la canalización, cada uno emitiendo un artefacto (JSON/SARIF) que el SOC ingiere:

Control	Herramienta	Qué detecta
SAST	Semgrep, Bandit	Vulnerabilidades en el código
SCA	Trivy fs, osv-scanner	Dependencias con CVE
Secretos	gitleaks	Credenciales en el repositorio
IaC	Checkov, tfsec	Mala configuración de infraestructura
Imagen	Trivy image	Vulnerabilidades en la imagen de contenedor

2.4 Bucles de retroalimentación (RA5.g)

- Revisión de código como control de calidad y de seguridad.
- Métricas DORA (frecuencia de despliegue, tiempo de entrega, tasa de fallo, tiempo de recuperación).
- Retrospectivas que convierten incidentes en mejoras del proceso.

3. Laboratorio guiado

> Entorno: repositorio del proyecto. Canalización de referencia en `PPS/material/pipeline/` (GitHub Actions y equivalente GitLab CI).

Parte A — Proteger la rama y el flujo de revisión (RA5.b)

1. Configura la protección de la rama principal: revisión obligatoria, CI requerido, prohibido el empuje directo.
2. Documenta el flujo: rama de trabajo → PR → revisión por par → CI en verde → fusión.

Parte B — Canalización de seguridad (RA5.a, RA5.c)

1. Adapta la canalización de `PPS/material/pipeline/ci-seguridad.yml` al proyecto: SAST (Semgrep), SCA (Trivy), secretos (gitleaks), IaC (Checkov). Cada trabajo emite SARIF.

```
# Ejecutar los mismos controles en local antes de subir
semgrep --config p/owasp-top-ten --sarif -o semgrep.sarif .
trivy fs --format sarif -o trivy.sarif .
gitleaks detect --report-format sarif --report-path gitleaks.sarif
checkov -d . -o sarif > checkov.sarif
```

Parte C — Demostrar el bloqueo (RA5.c, RA5.g)

1. Introduce **deliberadamente** un secreto de prueba y una dependencia vulnerable conocida en una rama, abre un PR y demuestra que la canalización **falla y bloquea** la fusión.
2. Revierte, deja la rama principal limpia y documenta la evidencia del bloqueo.

4. Entregables

- **`PPS-E12`** (reto integrado, rúbrica R3): canalización de integración continua ejecutable con al menos cuatro controles de seguridad y artefactos SARIF.
- **`PPS-E13`** (laboratorio, rúbrica R1): evidencia de que la canalización bloquea automáticamente ante un secreto y ante una dependencia vulnerable.

5. Criterios de superación

- La canalización **corre** en el repositorio (se ejecuta en cada PR), no es un fichero de ejemplo.
- La rama principal está protegida: no se puede fusionar con CI en rojo.
- El bloqueo se demuestra con un caso real (secreto + dependencia vulnerable).

6. Errores frecuentes

- Configurar los controles en modo aviso y no en modo bloqueo: informan pero no impiden nada.
- Proteger la rama pero permitir que el propietario del repositorio se salte la revisión.
- Emitir SARIF que nadie ingiere: el artefacto debe llegar a la gestión de vulnerabilidades del SOC.

7. Referencias

- OWASP **DevSecOps Maturity Model** — dsomm.owasp.org
- NIST SP 800-218 (SSDF).
- Semgrep, Trivy, gitleaks, Checkov — documentación oficial.
- Estado del arte DORA (DevOps Research and Assessment).

Ficha PPS-E12 — Rama principal protegida + flujo de revisión

UT7 · RA5.b · Instrumento I1 · Rúbrica R1 · Semana 15 Entrega: captura de la configuración + `NN-PPS-E12-flujo.md`.

Objetivo

Que en el repositorio del proyecto **nada entre a `main` sin revisión y sin la CI en verde**, y documentar el flujo de trabajo que lo garantiza.

Qué tienes que hacer

1. En el repositorio del proyecto (GitHub/GitLab de aula), configura la **protección de la rama principal** con, al menos: - Revisión obligatoria por par (mínimo 1 aprobación) antes de fusionar. - Comprobaciones de estado requeridas: el workflow de seguridad debe pasar en verde. - **Prohibido el empuje directo** a `main` (todo entra por Pull/Merge Request). - La protección **también aplica a administradores** (que el dueño no se la salte).
2. Crea una rama de trabajo, haz un cambio pequeño y llévalo hasta `main` **por el flujo correcto**: rama → PR → revisión → CI en verde → fusión.
3. **Comprueba que la barrera existe**: intenta un `git push` directo a `main` y captura el rechazo del servidor.

Qué entregas

- Captura de la pantalla de configuración de protección de rama (con las opciones activas).
- Captura del **rechazo** del push directo a `main`.
- `NN-PPS-E12-flujo.md`: describe el flujo rama→PR→revisión→CI→fusión de **tu** repositorio, con el nombre real de las ramas, quién revisa y qué comprobación se exige.

Se valora (R1, resumen)

- **Verificación del resultado**: no basta con activar la casilla; hay que **demostrar que bloquea** (el push rechazado). Marcar la opción sin probarla es "gate instalado, no activo".
- **Evidencia y reproducibilidad**: cualquiera debe poder repetir la configuración leyendo tu doc.

Pista

El push directo debe devolver algo parecido a:

```
! [remote rejected] main -> main (protected branch hook declined)
error: failed to push some refs to '...'
```

Si tu push a `main` **funciona**, la protección no está activa: revísala.

Ficha PPS-E13 — La canalización que BLOQUEA

UT7 · RA5 a,c,g · Instrumento I3 · Rúbrica R3 (reto integrado) · Semana 15 Entrega: PR en el repo + captura del build **en rojo** que bloquea la fusión.

Objetivo

Montar en el repositorio del proyecto una canalización de integración continua con **cuatro controles de seguridad** (SAST, SCA, secretos, IaC) configurada en **modo bloqueo**, y demostrar con un **caso que debe fallar** que impide la fusión. No vale que avise: tiene que **impedir**.

Punto de partida

- Mini-servicio vulnerable: `portal-mini/` (SQLi + dependencias con CVE + Dockerfile/Terraform mal configurados). Es lo que los controles deben detectar.
- Canalización de referencia (modo aviso): `~/xest-lab/2026-2027/PPS/material/pipeline/ci-seguridad.yml`.
- Canalización endurecida (modo bloqueo): `ci/ci-seguridad-bloquea.yml`.
- Regla Semgrep propia del curso: `semgrep-reglas/sqli-concat.yml`.

Parte A — Ensaya los controles en LOCAL (antes de subir)

```
cd portal-mini
semgrep --config p/owasp-top-ten --config ../semgrep-reglas/sqli-concat.yml --sarif -o
semgrep.sarif .
trivy fs --format sarif -o trivy.sarif .
gitleaks detect --source . --report-format sarif --report-path gitleaks.sarif --redact
checkov -d . --output sarif --output-file-path .
```

Abre cada SARIF y localiza **qué** encontró cada herramienta y **en qué fichero/línea**. Anótalo: esos artefactos son los que el SOC de Incidentes (IC) ingiere.

Parte B — Monta la canalización en modo bloqueo

1. Copia `ci/ci-seguridad-bloquea.yml` al repo como `.github/workflows/ci-seguridad.yml`.
2. Comprueba que **ningún** control lleva `|| true` ni `soft-fail`: un hallazgo debe dar exit 1.
3. La rama `main` (PPS-E12) exige que este workflow pase en verde para poder fusionar.

Parte C — El caso que DEBE fallar

1. Crea una rama, añade `caso-que-debe-fallar/config-con-secreto.py` (secreto **falso**) y, opcionalmente, deja las dependencias con CVE de `portal-mini/requirements.txt`.
2. Abre un PR contra `main`.
3. **Comprueba y captura** que la CI se pone en **rojo** (falla al menos el job de secretos y el de SCA) y que **la fusión queda bloqueada**.
4. Revierte: quita el secreto, deja la rama principal limpia y documenta la evidencia del bloqueo.

Qué entregas

- El PR con el workflow en el repositorio (ejecutándose en cada PR, no un fichero de ejemplo).
- Captura del **build en rojo** con la fusión bloqueada + qué control lo paró.
- Breve nota: qué encontró cada control en local (Parte A) y dónde llegan los SARIF (SOC de IC).

Se valora (R3, resumen)

- **Automatización y repetibilidad**: la CI corre sola en cada PR y es reproducible desde cero.
- **Integración con el SOC**: los SARIF llegan a la gestión de vulnerabilidades de IC.
- **Prueba del bloqueo**: la evidencia decisiva es el **caso que falla**, no la CI en verde. Una canalización que solo has visto pasar en verde **no está probada** como barrera.

PPS · UT8 — Despliegue automatizado, resiliencia y recuperación ante desastres

5 horas · RA5 (d, e, f) · Servicio SOC: Prevención — continuidad del despliegue · Evidencias PPS-E14, PPS-E15

Qué vamos a conseguir

Desplegar, revertir y recuperar con medidas reales.

Antes de empezar

Herramientas: Docker, nginx, Python y SQLite; scripts del laboratorio.

Consulta la guía de instalación, la guía de herramientas y las precisiones del curso. Prepara una carpeta propia para resultados; conserva intactos los datos de partida.

Ruta de trabajo

1. Lee README.md y genera el dataset sintético con el comando indicado. Guarda una copia inicial.
2. Arranca solo el servicio estable y comprueba /health y una operación funcional.
3. Sigue ejercicio-E14-canario.md: ejecuta el canario degradado y conserva la decisión de reversión.
4. Repite con canario sano. Distingue respuesta del proxy y respuesta directa de cada versión.
5. Sigue ejercicio-E15-dr.md en datos de laboratorio: realiza copia, verifica integridad, simula pérdida y restaura.
6. Mide inicio de interrupción, retorno funcional y antigüedad del último dato recuperado. Calcula RTO/RPO con esas marcas y prueba que los datos restaurados son utilizables.

Cómo comprobar que está hecho

Reversión observada; restauración real; tiempos y pérdida de datos medidos.

> docker compose down detiene y elimina contenedores/redes, pero no equivale a restaurar los datos.

Desarrollo y fichas

- Explicación completa de la unidad.
- README
- procedimiento-dr-plantilla
- ejercicio-E14-canario
- ejercicio-E15-dr

Entrega y revisión

Los nombres y formatos están en el cuaderno de entregables. Antes de entregar, comprueba alcance, pasos reproducibles, resultados, interpretación y límites. Identifica tu trabajo por número de lista, módulo y evidencia.

Incluye una dificultad que hayas resuelto y explica por qué la solución funciona. Si utilizas una salida de ejemplo, declárala; no la presentes como ejecución propia.

PPS · UT8 — Despliegue automatizado, resiliencia y recuperación ante desastres

5 horas · RA5 (d, e, f) · Servicio SOC: Prevención — continuidad del despliegue · Evidencias `PPS-E14`, `PPS-E15`

---

1. Objetivo de la unidad

Cierra el módulo: llevar el software a producción de forma que un fallo no se convierta en una caída del servicio, y tener un procedimiento de recuperación **probado**, no supuesto. El criterio: sin reversión probada, no se libera; sin recuperación ensayada, no hay plan.

2. Contenido

2.1 Estrategias de despliegue (RA5.d)

Estrategia	Cómo funciona	Cuándo
Azul-verde	Dos entornos idénticos; se conmuta el tráfico	Servicios sin estado

Estrategia	Cómo funciona	Cuándo
Canario	Se envía un % del tráfico a la versión nueva	Validar con tráfico real antes de generalizar
Indicadores de funcionalidad	Desacoplan despliegue de activación	Activar/desactivar sin re-desplegar
Reversión	Volver a la versión previa buena	Siempre: probada antes de liberar

2.2 Resiliencia y autorreparación (RA5.e)

- Gestión automatizada de configuración y orquestación de contenedores.
- Sondas de vida y disponibilidad (liveness/readiness) que reinician lo que no responde.
- Escalado automático ante carga.
- **Ingeniería del caos**: inyectar fallos controlados para verificar que el sistema se recupera solo.

2.3 Recuperación ante desastres (RA5.f)

- **RTO** (Recovery Time Objective): cuánto puede tardar en volver el servicio.
- **RPO** (Recovery Point Objective): cuánto dato se puede permitir perder.
- Copias verificadas (una copia que no se restaura no es una copia).
- Procedimiento escrito **y ensayado**, con RTO/RPO medidos en el ensayo, no estimados.

3. Laboratorio guiado

Parte A — Despliegue canario con reversión (RA5.d, RA5.e)

1. Despliega el portal del SOC en modo canario (p. ej. dos réplicas y enrutado por peso, o dos servicios y conmutación).
2. Configura una sonda de disponibilidad y una condición de reversión automática ante degradación (código 5xx por encima de un umbral).
3. Provoca la degradación en la versión canaria y demuestra la **reversión automática**.

```
# Ejemplo de sonda de disponibilidad en Docker Compose
# healthcheck:
#   test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
#   interval: 10s
#   retries: 3
docker compose ps          # ver estado de salud
docker compose logs -f app # observar reinicio/reversión
```

Parte B — Inyección de fallo y medición (RA5.e)

1. Inyecta un fallo controlado (detén un contenedor, satura CPU) y **mide** el tiempo de recuperación real.

Parte C — Recuperación ante desastres (RA5.f)

1. Redacta el procedimiento de recuperación ante desastres del servicio (qué se restaura, en qué orden, desde qué copia).
2. **Ensaya** una restauración completa desde copia y mide RTO y RPO reales. Documenta la desviación respecto a los objetivos.

4. Entregables

- **`PPS-E14`** (reto integrado, rúbrica R3): plan de despliegue con reversión probada y evidencia del ensayo (degradación → reversión automática).
- **`PPS-E15`** (informe, rúbrica R2): procedimiento de recuperación ante desastres con RTO y RPO **medidos** en el ensayo, no estimados.

5. Criterios de superación

- La reversión se demuestra funcionando ante una degradación real.
- La recuperación se ensaya de verdad y los RTO/RPO son medidos.
- El procedimiento es reproducible por otra persona con solo el documento.

6. Errores frecuentes

- Declarar RTO/RPO sin haber restaurado nunca: son deseos, no objetivos.
- Confiar en copias que nunca se han restaurado.
- Reversión "manual" no documentada: en un incidente real no está disponible.

7. Referencias

- Google SRE Book — capítulos de despliegue y gestión de la carga.
- NIST SP 800-34 — Contingency Planning Guide (RTO/RPO).
- OWASP DevSecOps Maturity Model — despliegue y operación.

PPS · UT8 — Laboratorio: despliegue canario, resiliencia y DR

Material de manos para las dos sesiones de UT8. Todo corre en la **VLAN de aula aislada**; nada se expone a Internet. Solo biblioteca estándar de Python + Docker + sqlite3.

Piezas

Fichero	Para qué
`portal/app.py`	Portal SOC mínimo con `/health` que comprueba **de verdad** su dependencia (la BD).
`datos/seed_alertas.py`	Genera un dataset **sintético** de alertas del SOC (sin datos reales).
`docker-compose.canario.yml`	Dos versiones del portal (`estable` v1.0.0 / `canario` v2.0.0) + proxy de reparto.
`nginx-canario.conf`	Reparto por peso 90/10 y reintento al estable si el canario da 5xx.
`deploy-canario.sh`	Despliegue canario con sonda y **reversión automática** → evidencia **PPS-E14**.
`sonda.sh`	Sonda de disponibilidad reutilizable (URL, muestras, intervalo).
`caos.sh`	Inyección de fallo (SIGKILL) y **MTTR medido** por la política de reinicio.
`dr/backup.sh`	Copia **verificada** (integrity_check) de la BD.
`dr/restore.sh`	Restaura la última copia buena.
`dr/medir-rto-rpo.sh`	Ensayo DR completo con **RTO y RPO medidos** → evidencia **PPS-E15**.
`dr/procedimiento-dr-plantilla.md`	Plantilla del procedimiento que rellena el alumnado.

Puesta en marcha (una vez, antes de clase)

```
cd PPS/UT8/laboratorio
python3 datos/seed_alertas.py datos/soc.db 500        # crea el dataset sintético
```

```
docker compose -f docker-compose.canario.yml up -d estable
curl -s http://localhost:8091/health                 # debe dar {"status":"ok",...}
```

Sesión 1 — canario y caos

```
CANARY_FAIL=http5xx ./deploy-canario.sh   # canario roto -> REVERSIÓN AUTOMÁTICA (PPS-E14)
./deploy-canario.sh                        # canario sano -> promoción
./caos.sh                                  # mata el portal y mide el MTTR real
```

Sesión 2 — recuperación ante desastres

```
./dr/backup.sh                             # copia verificada
./dr/medir-rto-rpo.sh                      # desastre + restauración + RTO/RPO medidos (PPS-E15)
```

Puertos

- `8091` → portal **estable** (acceso directo).
- `8092` → portal **canario** (acceso directo, solo durante el despliegue).
- `8080` → **proxy** de reparto (solo con `--profile proxy`).

Nota de seguridad de aula

Los objetivos son deliberadamente frágiles y las IP de los datos son de rangos de documentación (RFC 5737) o privados (RFC 1918). **No enrutar fuera de la VLAN aislada.**

Procedimiento de recuperación ante desastres — portal SOC de xest

> Plantilla que rellena el alumnado para **PPS-E15**. Debe ser reproducible por otra persona con solo este documento. Los RTO/RPO se rellenan con los valores **medidos en el ensayo**, no con estimaciones.

1. Alcance

- Servicio: portal SOC de xest (`portal-estable`, BD de alertas `datos/soc.db`).

- Desastre cubierto: pérdida de la base de datos de alertas.

2. Objetivos declarados (antes del ensayo)

- RTO objetivo: _____ (lo que la organización se propone).
- RPO objetivo: _____ .

3. Qué se restaura, en qué orden, desde qué copia

1. Copia de origen: última `backups/soc-*.db` con `integrity_check = ok`.
2. Orden de restauración:
3. Restaurar el fichero de BD.
4. Reiniciar el contenedor del portal.
5. Verificar `/health = 200`.
6. Verificación de éxito: `curl -s http://localhost:8091/health` devuelve `status: ok`.

4. Ejecución del ensayo (comandos)

```
./dr/backup.sh                # copia verificada de partida
./dr/medir-rto-rpo.sh          # desastre controlado + restauración + medición
```

5. Resultado MEDIDO en el ensayo

Métrica	Objetivo	Medido	Desviación
RTO (s)	_____	_____	_____
RPO (s)	_____	_____	_____
Datos perdidos (alertas)	—	_____	—

6. Conclusión y mejoras

- ¿Se cumplió el RTO/RPO? _____
- Acción para reducir el RPO (p. ej. copias más frecuentes): _____
- Acción para reducir el RTO (p. ej. restauración automatizada): _____

PPS-E14 · Despliegue canario con reversión automática (alumnado)

RA5 d,e · rúbrica R3 (reto integrado). Entrega: `NN-PPS-E14-despliegue.md` + el log del despliegue.

Contexto

Vais a liberar la versión 2.0.0 del portal SOC. Regla del módulo: **sin reversión probada, no se libera**. Lo demostráis provocando una degradación y viendo que el sistema **revierte solo**.

Antes de empezar

```
cd PPS/UT8/laboratorio
python3 datos/seed_alertas.py datos/soc.db 500
docker compose -f docker-compose.canario.yml up -d estable
curl -s http://localhost:8091/health          # {"status":"ok", ...}
```

Tareas

1. **Sonda que de verdad mide.** Explica en tu entrega por qué `/health` de este portal **no** puede quedarse siempre en "healthy": ¿qué comprueba antes de responder 200? Demuéstralo parando la dependencia (pista: `FAIL_MODE=db`).
2. **Canario degradado → reversión automática.** Lanza el despliegue de un canario roto y captura el log donde se ve la reversión: `bash CANARY_FAIL=http5xx ./deploy-canario.sh` El log `deploy-*.log` es tu evidencia. Debe verse: sondas en FALLO, tasa de error por encima del umbral y la línea `REVERSIÓN AUTOMÁTICA`.
3. **Canario sano → promoción.** Repite con un canario bueno y muestra que ahora **se promueve**: `bash ./deploy-canario.sh`
4. **Redacta** en `NN-PPS-E14-despliegue.md`: estrategia usada (canario), umbral de reversión elegido y por qué, y qué pasaría en producción con el proxy de reparto por peso.

Criterio de superación

- La reversión se demuestra **funcionando ante una degradación real** (no descrita: ejecutada).
- El log adjunto prueba la secuencia sonda → umbral superado → reversión.
- Distingues "el gate está instalado" de "el gate está activo": lo pruebas con un caso que **debe** fallar.

PPS-E15 · Recuperación ante desastres ensayada (alumnado)

RA5 f · rúbrica R2 (informe). Entrega: `NN-PPS-E15-dr.md` (con el procedimiento y los valores **medidos**) + el volcado de la ejecución.

Contexto

Regla del módulo: **una copia que no se restaura no es una copia**, y **sin recuperación ensayada no hay plan**. Los RTO/RPO se **miden en el ensayo**, no se estiman.

Antes de empezar

```
cd PPS/UT8/laboratorio
python3 datos/seed_alertas.py datos/soc.db 500     # si no lo hiciste ya
docker compose -f docker-compose.canario.yml up -d estable
```

Tareas

1. **Copia verificada.** Ejecuta `./dr/backup.sh` y explica por qué no basta con que el fichero exista: ¿qué comprobación adicional hace el script y por qué importa?
2. **Ensayo completo.** Ejecuta el ensayo y captura toda la salida: `bash ./dr/medir-rto-rpo.sh | tee mi-ensayo-dr.txt` Anota **RTO medido**, **RPO medido** y **datos perdidos**.
3. **Procedimiento reproducible.** Rellena `dr/procedimiento-dr-plantilla.md` con tus valores (objetivos vs medidos) y súbelo como parte de la entrega.
4. **Análisis.** ¿Se cumplieron los objetivos? Propón **una** mejora para reducir el RPO y **una** para reducir el RTO, y di qué tendrías que cambiar en el laboratorio para lograrlas.

Criterio de superación

- La restauración se ensaya **de verdad**: el servicio vuelve a `/health = 200` tras el desastre.
- RTO y RPO son **medidos** (salen del cronómetro y del recuento), con su desviación respecto al objetivo.
- El procedimiento es reproducible por otra persona **solo con el documento**.